

```
{
  "openapi": "3.0.2",
  "info": {
    "title": "Bond Local API",
    "version": "v3.0.0",
    "x-logo": {
      "url": "https://s3.amazonaws.com/docs-cloud.appbond.com/bond.png"
    }
  },

```

Introduction

Scope

The Bond V2 API allows control of Ceiling Fans, Fireplaces, and other Bond devices. It is intended for integration with offline control systems, for use by advanced users, hobbyists, integrators, and installers.

For the purposes of device control, it does not matter whether these devices are:

- Remote-controlled devices that *Work with Bond* (connected via a Bond Bridge)
- Smart devices that are *Smart by Bond*

This documentation describes the Local HTTP API for Bond products running v2 firmware.

- Does **not** require an internet connection
- Requires the API client to communicate directly with the Bond over HTTP
- Usually means being on the same Wi-Fi network

A Cloud API is also being developed and will be very similar to the Local API.

Contributing

We welcome corrections and improvements to this documentation!

Source repository:

<https://github.com/bondhome/api-v2>

Feel free to open a Pull Request.

Security

The Bond Local API uses unencrypted HTTP with a simple token-based authentication mechanism.

Why HTTP and not HTTPS?

After speaking with many users interested in the Local API, we discovered:

- Users were satisfied with Wi-Fi password protection.
- Low-latency local control was more important than HTTPS.
- HTTPS is difficult to implement securely on offline local devices.

+

The standard web certificate chain-of-trust does not work offline because:

- Bond devices do not have public domain names.
- Browsers would not trust self-signed certificates.

If you have untrusted users on your network, Bond recommends placing devices on a separate home automation network.

Security on the Internet

When Bond products communicate with Bond Cloud:

- TLS encryption is used.
- Per-device public key cryptography is used.
- No unencrypted communication occurs between Bond hardware and Bond Cloud.

> Always place Bond behind a firewall and do **not** expose the Local API to the public internet

Firmware Versions

To get the best match between this documentation and your Bond's exposed API, update your Bond to the latest firmware version.

Some features may require Beta firmware.

- Beta apps: <https://forum.bondhome.io/t/bond-local-api-public-beta-link/589>
- Beta forum: <https://forum.bondhome.io/c/beta/5>

```
},
"servers": [
  {
    "url": "https://192.168.0.100/",
    "description": "Local API"
  }
],
"paths": {
  "/v2/debug/rfman": {
    "get": {
      "responses": {
        "200": {
          "content": {
            "application/json": {
              "schema": {
                "$ref": "#/components/schemas/Schema"
              }
            }
          }
        },
        "description": "RF manager configuration"
      },
      "summary": "Get RF Manager configuration",
      "description": "(Bond Bridge only)\nIntended for use in debugging signal generation. All val
      "tags": [
        "RF Manager"
      ]
    },
    "patch": {
      "requestBody": {
        "content": {
          "application/json": {
            "schema": {
              "$ref": "#/components/schemas/Schema"
            }
          }
        }
      },
      "responses": {
        "200": {
          "description": "RF Manager configured",
          "content": {
            "application/json": {
              "schema": {
                "$ref": "#/components/schemas/Schema"
              }
            }
          }
        },
        "400": {
          "description": "Error parsing request"
        }
      },
      "summary": "Configure RF Manager",
      "description": "(Bond Bridge only)\nIntended for use in debugging signal generation. All val
      "tags": [
        "RF Manager"
      ]
    },
    "delete": {
      "responses": {
        "204": {
          "description": "RF Manager settings reset"
        }
      },
      "summary": "Reset RF Manager settings",
      "tags": [
        "RF Manager"
      ]
    }
  }
}
```

```

    },
    "/v2/debug/leds": {
      "get": {
        "responses": {
          "200": {
            "content": {
              "application/json": {
                "schema": {
                  "$ref": "#/components/schemas/Leds"
                }
              }
            }
          },
          "description": "LED state"
        },
        "summary": "Get LED state",
        "description": "Get status from the BLEED driver.\n\nIf `manual` is `0`, `value` reflects the",
        "tags": [
          "LEDs"
        ]
      },
      "patch": {
        "requestBody": {
          "content": {
            "application/json": {
              "schema": {
                "$ref": "#/components/schemas/Leds"
              }
            }
          }
        },
        "responses": {
          "200": {
            "description": "LED state was changed successfully"
          },
          "400": {
            "description": "Error parsing request"
          },
          "500": {
            "description": "Driver error"
          }
        },
        "summary": "Manually control LEDs",
        "description": "Use this endpoint to manually control LEDs, such as during factory test.\nBe",
        "tags": [
          "LEDs"
        ]
      }
    },
    "/v2/debug/livelog": {
      "get": {
        "responses": {
          "200": {
            "content": {
              "application/json": {
                "schema": {
                  "$ref": "#/components/schemas/LiveLog"
                }
              }
            }
          },
          "description": "LiveLog configuration"
        },
          "204": {
            "description": "LiveLog disabled"
          }
        },
        "summary": "Get LiveLog configuration",
        "description": "Get the current configuration of the LiveLog component.\n\nReturns `204 No C",
        "tags": [
          "LiveLog"
        ]
      },
      "put": {
        "requestBody": {
          "content": {
            "application/json": {
              "schema": {

```

```

        "$ref": "#/components/schemas/LiveLog"
      }
    }
  },
  "responses": {
    "200": {
      "description": "LiveLog configured"
    },
    "400": {
      "description": "Request rejected. The `_error_msg` field indicates the cause:\n\n- `_ip`"
    },
    "500": {
      "description": "[Added in v4.10.0] Failed to derive the encryption key or encrypt\nthe f"
    }
  },
  "summary": "Configure LiveLog",
  "description": "Instructs Bond to stream log output as UDP datagrams to the specified\nIP add",
  "tags": [
    "LiveLog"
  ]
},
"delete": {
  "responses": {
    "204": {
      "description": "LiveLog disabled"
    }
  },
  "summary": "Disable LiveLog",
  "description": "Stops the LiveLog stream and clears the stored `ip`, `port`, and\n`keyphrase",
  "tags": [
    "LiveLog"
  ]
}
},
"/v2/debug/wifi": {
  "get": {
    "responses": {
      "200": {
        "content": {
          "application/json": {
            "schema": {
              "$ref": "#/components/schemas/Wifi"
            }
          }
        }
      },
      "description": "Wi-Fi power settings"
    },
    "summary": "Get Wi-Fi power settings",
    "description": "The `shutdown` field indicates the power state of the Wi-Fi (+BLE) radio:\n",
    "tags": [
      "WiFi"
    ]
  },
  "patch": {
    "requestBody": {
      "content": {
        "application/json": {
          "schema": {
            "$ref": "#/components/schemas/Wifi"
          }
        }
      }
    },
    "responses": {
      "200": {
        "description": "Wi-Fi power settings changed"
      }
    },
    "summary": "Change Wi-Fi power settings",
    "description": "Set `shutdown` to `1` or `0` to control the Wi-Fi (+BLE) radio power level. ",
    "tags": [
      "WiFi"
    ]
  }
}
},

```

```

"/v2/debug/beau/{partition}": {
  "parameters": [
    {
      "in": "path",
      "name": "partition",
      "required": true,
      "schema": {
        "type": "string"
      }
    }
  ],
  "get": {
    "responses": {
      "200": {
        "content": {
          "application/json": {
            "schema": {
              "$ref": "#/components/schemas/BeauDb"
            }
          }
        }
      }
    },
    "description": "Database statistics"
  },
  "summary": "Get database statistics",
  "description": "Get some statistics about the main Bond database such as free space available",
  "tags": [
    "Database"
  ]
},
"/v2/bridge": {
  "get": {
    "responses": {
      "200": {
        "content": {
          "application/json": {
            "schema": {
              "$ref": "#/components/schemas/Bridge"
            }
          }
        }
      },
      "401": {
        "$ref": "#/components/responses/Unauthorized"
      },
      "404": {
        "$ref": "#/components/responses/NotFound"
      },
      "500": {
        "$ref": "#/components/responses/InternalServerError"
      }
    },
    "security": [
      {
        "OAuth": [
          "oauth2"
        ]
      }
    ],
    "summary": "Get Bridge info",
    "tags": [
      "Bridge"
    ]
  },
  "patch": {
    "requestBody": {
      "content": {
        "application/json": {
          "schema": {
            "$ref": "#/components/schemas/Bridge"
          }
        }
      }
    },
    "responses": {

```

```

        "200": {
          "content": {
            "application/json": {
              "schema": {
                "$ref": "#/components/schemas/Bridge"
              }
            }
          },
          "description": "Change Bridge info"
        },
        "401": {
          "$ref": "#/components/responses/Unauthorized"
        },
        "404": {
          "$ref": "#/components/responses/NotFound"
        },
        "500": {
          "$ref": "#/components/responses/InternalServerError"
        }
      },
      "security": [
        {
          "BasicAuth": []
        }
      ],
      "summary": "Change Bridge info",
      "tags": [
        "Bridge"
      ]
    },
    "delete": {
      "responses": {
        "204": {
          "description": "Reset Bridge info"
        },
        "401": {
          "$ref": "#/components/responses/Unauthorized"
        },
        "404": {
          "$ref": "#/components/responses/NotFound"
        },
        "500": {
          "$ref": "#/components/responses/InternalServerError"
        }
      },
      "security": [
        {
          "BasicAuth": []
        }
      ],
      "summary": "Reset Bridge info",
      "description": "After deletion, default values will be restored.\n",
      "tags": [
        "Bridge"
      ]
    }
  },
  "/v2/sys/locale": {
    "get": {
      "responses": {
        "200": {
          "content": {
            "application/json": {
              "schema": {
                "$ref": "#/components/schemas/Locale"
              }
            }
          },
          "description": "Get Locale info"
        },
        "401": {
          "$ref": "#/components/responses/Unauthorized"
        },
        "404": {
          "$ref": "#/components/responses/NotFound"
        },
        "500": {

```

```

        "$ref": "#/components/responses/InternalServerError"
    },
    },
    "security": [
        {
            "OAuth": [
                "oauth2"
            ]
        }
    ],
    "summary": "Get Locale info",
    "tags": [
        "Locale"
    ]
},
"patch": {
    "requestBody": {
        "content": {
            "application/json": {
                "schema": {
                    "$ref": "#/components/schemas/Locale"
                }
            }
        }
    },
    "responses": {
        "200": {
            "content": {
                "application/json": {
                    "schema": {
                        "$ref": "#/components/schemas/Locale"
                    }
                }
            },
            "description": "Change Locale info"
        },
        "401": {
            "$ref": "#/components/responses/Unauthorized"
        },
        "404": {
            "$ref": "#/components/responses/NotFound"
        },
        "500": {
            "$ref": "#/components/responses/InternalServerError"
        }
    },
    "security": [
        {
            "BasicAuth": []
        }
    ],
    "summary": "Change Locale info",
    "tags": [
        "Locale"
    ]
},
"delete": {
    "responses": {
        "204": {
            "description": "Reset Locale info"
        },
        "401": {
            "$ref": "#/components/responses/Unauthorized"
        },
        "404": {
            "$ref": "#/components/responses/NotFound"
        },
        "500": {
            "$ref": "#/components/responses/InternalServerError"
        }
    },
    "security": [
        {
            "BasicAuth": []
        }
    ],
    "summary": "Reset Locale info",

```

```

        "description": "After deletion, default values will be restored.\n",
        "tags": [
            "Locale"
        ]
    },
    "/v2/devices": {
        "get": {
            "description": "Returns a list of device IDs and the corresponding device hashes.\nThe hashes",
            "responses": {
                "200": {
                    "content": {
                        "application/json": {
                            "schema": {
                                "$ref": "#/components/schemas/DeviceList"
                            }
                        }
                    },
                    "description": "List active devices"
                },
                "401": {
                    "$ref": "#/components/responses/Unauthorized"
                },
                "500": {
                    "$ref": "#/components/responses/InternalServerError"
                }
            },
            "security": [
                {
                    "OAuth": [
                        "oauth2"
                    ]
                }
            ],
            "summary": "List your devices",
            "tags": [
                "Devices"
            ]
        },
        "post": {
            "requestBody": {
                "content": {
                    "application/json": {
                        "schema": {
                            "$ref": "#/components/schemas/Device"
                        }
                    }
                }
            },
            "responses": {
                "201": {
                    "$ref": "#/components/responses/Created"
                },
                "400": {
                    "$ref": "#/components/responses/BadRequest"
                },
                "401": {
                    "$ref": "#/components/responses/Unauthorized"
                },
                "409": {
                    "$ref": "#/components/responses/Conflict"
                },
                "500": {
                    "$ref": "#/components/responses/InternalServerError"
                }
            },
            "summary": "Create new device",
            "tags": [
                "Devices"
            ]
        }
    },
    "/v2/devices/{device_id}": {
        "delete": {
            "responses": {
                "204": {
                    "description": "Remove device"
                }
            }
        }
    }
}

```



```

    },
    "401": {
      "$ref": "#/components/responses/Unauthorized"
    },
    "404": {
      "$ref": "#/components/responses/NotFound"
    },
    "500": {
      "$ref": "#/components/responses/InternalServerError"
    }
  },
  "security": [
    {
      "BasicAuth": []
    }
  ],
  "summary": "Delete device",
  "tags": [
    "Devices"
  ]
},
"get": {
  "description": "Basic device information. The `\"_\"` in each of `state`, `properties`, `n`ac",
  "responses": {
    "200": {
      "content": {
        "application/json": {
          "schema": {
            "$ref": "#/components/schemas/Device"
          }
        }
      },
      "description": "Get device"
    },
    "401": {
      "$ref": "#/components/responses/Unauthorized"
    },
    "404": {
      "$ref": "#/components/responses/NotFound"
    },
    "500": {
      "$ref": "#/components/responses/InternalServerError"
    }
  },
  "security": [
    {
      "OAuth": [
        "oauth2"
      ]
    }
  ],
  "summary": "Get device",
  "tags": [
    "Devices"
  ]
},
"parameters": [
  {
    "in": "path",
    "name": "device_id",
    "required": true,
    "schema": {
      "type": "number"
    }
  }
],
"patch": {
  "requestBody": {
    "required": true,
    "content": {
      "application/json": {
        "schema": {
          "$ref": "#/components/schemas/Device"
        }
      }
    }
  }
},

```

```

"responses": {
  "200": {
    "content": {
      "application/json": {
        "schema": {
          "$ref": "#/components/schemas/Device"
        }
      }
    },
    "description": "Change device field"
  },
  "401": {
    "$ref": "#/components/responses/Unauthorized"
  },
  "404": {
    "$ref": "#/components/responses/NotFound"
  },
  "500": {
    "$ref": "#/components/responses/InternalServerError"
  }
},
"security": [
  {
    "BasicAuth": []
  }
],
"summary": "Change device field",
"tags": [
  "Devices"
]
},
"/v2/devices/{device_id}/state": {
  "get": {
    "responses": {
      "200": {
        "content": {
          "application/json": {
            "schema": {
              "$ref": "#/components/schemas/State"
            }
          }
        },
        "description": "Get device"
      },
      "401": {
        "$ref": "#/components/responses/Unauthorized"
      },
      "404": {
        "$ref": "#/components/responses/NotFound"
      },
      "500": {
        "$ref": "#/components/responses/InternalServerError"
      }
    },
    "security": [
      {
        "OAuth": [
          "oauth2"
        ]
      }
    ],
    "summary": "Get device state",
    "tags": [
      "Device State"
    ]
  },
  "patch": {
    "requestBody": {
      "content": {
        "application/json": {
          "schema": {
            "$ref": "#/components/schemas/State"
          }
        }
      }
    }
  }
},

```

```

"responses": {
  "200": {
    "description": "State belief updated"
  },
  "401": {
    "$ref": "#/components/responses/Unauthorized"
  },
  "404": {
    "$ref": "#/components/responses/NotFound"
  },
  "500": {
    "$ref": "#/components/responses/InternalServerError"
  }
},
"security": [
  {
    "OAuth": [
      "oauth2"
    ]
  }
],
"summary": "Update state belief",
"description": "Update the Bond's belief about the state of a device.\n\nThis is useful, for",
"tags": [
  "Device State"
]
},
"parameters": [
  {
    "in": "path",
    "name": "device_id",
    "required": true,
    "schema": {
      "type": "number"
    }
  }
]
},
"/v2/devices/{device_id}/addr": {
  "get": {
    "responses": {
      "200": {
        "content": {
          "application/json": {
            "schema": {
              "$ref": "#/components/schemas/RemoteAddr"
            }
          }
        }
      },
      "description": "Get SBB Remote Address"
    },
    "401": {
      "$ref": "#/components/responses/Unauthorized"
    },
    "404": {
      "$ref": "#/components/responses/NotFound"
    },
    "500": {
      "$ref": "#/components/responses/InternalServerError"
    }
  },
  "security": [
    {
      "OAuth": [
        "oauth2"
      ]
    }
  ],
  "summary": "Get SBB Remote Address and Learn Window",
  "description": "[SBB-only]\nProvides the current SBB remote address and information about the",
  "tags": [
    "Device Remote Address"
  ]
},
"patch": {
  "requestBody": {
    "content": {

```

```

        "application/json": {
            "schema": {
                "$ref": "#/components/schemas/RemoteAddr"
            }
        }
    },
    "responses": {
        "200": {
            "content": {
                "application/json": {
                    "schema": {
                        "$ref": "#/components/schemas/RemoteAddr"
                    }
                }
            },
            "description": "Modify Device Remote Address or Learn Window"
        },
        "401": {
            "$ref": "#/components/responses/Unauthorized"
        },
        "404": {
            "$ref": "#/components/responses/NotFound"
        },
        "500": {
            "$ref": "#/components/responses/InternalServerError"
        }
    },
    "security": [
        {
            "BasicAuth": []
        }
    ],
    "summary": "Modify Device Remote Address or Learn Window",
    "description": "[SBB-only]\nModify either the Device Remote Address or open/close the learn window",
    "tags": [
        "Device Remote Address"
    ]
},
"delete": {
    "responses": {
        "200": {
            "description": "Remote address successfully reset, learn window closed.",
            "content": {
                "application/json": {
                    "schema": {
                        "$ref": "#/components/schemas/RemoteAddr"
                    }
                }
            }
        },
        "401": {
            "$ref": "#/components/responses/Unauthorized"
        },
        "404": {
            "$ref": "#/components/responses/NotFound"
        },
        "500": {
            "$ref": "#/components/responses/InternalServerError"
        }
    },
    "security": [
        {
            "BasicAuth": []
        }
    ],
    "summary": "Reset Device Remote Address and Learn Window",
    "description": "[SBB-only]\nIf no `addr` is provided, then:\n Restores factory default Device Remote Address",
    "tags": [
        "Device Remote Address"
    ]
},
"parameters": [
    {
        "in": "path",
        "name": "device_id",
        "required": true,

```

```

        "schema": {
            "type": "number"
        }
    }
},
"/v2/devices/{device_id}/reload": {
    "put": {
        "responses": {
            "204": {
                "description": "Device definition reloaded"
            },
            "401": {
                "$ref": "#/components/responses/Unauthorized"
            },
            "404": {
                "$ref": "#/components/responses/NotFound"
            },
            "500": {
                "$ref": "#/components/responses/InternalServerError"
            }
        },
        "security": [
            {
                "OAuth": [
                    "oauth2"
                ]
            }
        ],
        "summary": "Reload device",
        "tags": [
            "Device Reload"
        ],
        "description": "Reload the command table for the given device definition. This deletes all\n",
        "parameters": [
            {
                "in": "path",
                "name": "device_id",
                "required": true,
                "schema": {
                    "type": "number"
                }
            }
        ]
    },
    "/v2/devices/{device_id}/properties": {
        "get": {
            "responses": {
                "200": {
                    "content": {
                        "application/json": {
                            "schema": {
                                "$ref": "#/components/schemas/Properties"
                            }
                        }
                    }
                },
                "description": "Get device properties"
            },
            "401": {
                "$ref": "#/components/responses/Unauthorized"
            },
            "404": {
                "$ref": "#/components/responses/NotFound"
            },
            "500": {
                "$ref": "#/components/responses/InternalServerError"
            }
        },
        "security": [
            {
                "OAuth": [
                    "oauth2"
                ]
            }
        ],
        "summary": "Get device properties",

```

```

        "description": "See the Features section above for other properties which may be available\n",
        "tags": [
            "Device Properties"
        ]
    },
    "patch": {
        "requestBody": {
            "content": {
                "application/json": {
                    "schema": {
                        "$ref": "#/components/schemas/Properties"
                    }
                }
            }
        },
        "responses": {
            "200": {
                "description": "Properties updated"
            },
            "401": {
                "$ref": "#/components/responses/Unauthorized"
            },
            "404": {
                "$ref": "#/components/responses/NotFound"
            },
            "500": {
                "$ref": "#/components/responses/InternalServerError"
            }
        },
        "security": [
            {
                "OAuth": [
                    "oauth2"
                ]
            }
        ],
        "summary": "Update properties",
        "description": "See the documentation of the individual properties for whether they can be P",
        "tags": [
            "Device Properties"
        ]
    },
    "parameters": [
        {
            "in": "path",
            "name": "device_id",
            "required": true,
            "schema": {
                "type": "number"
            }
        }
    ]
},
"/v2/devices/{device_id}/power_cycle_state": {
    "get": {
        "responses": {
            "200": {
                "content": {
                    "application/json": {
                        "schema": {
                            "$ref": "#/components/schemas/PowerCycleState"
                        }
                    }
                },
                "description": "Current Power Cycle State"
            },
            "401": {
                "$ref": "#/components/responses/Unauthorized"
            },
            "404": {
                "$ref": "#/components/responses/NotFound"
            },
            "500": {
                "$ref": "#/components/responses/InternalServerError"
            }
        },
        "security": [

```

```

        {
          "OAuth": [
            "oauth2"
          ]
        },
        "summary": "Read Power Cycle State",
        "description": "This feature is intended to be used with smart Ceiling Fan receivers\nthat a",
        "tags": [
          "Power Cycle State"
        ]
      },
      "patch": {
        "requestBody": {
          "content": {
            "application/json": {
              "schema": {
                "$ref": "#/components/schemas/PowerCycleState"
              }
            }
          }
        },
        "responses": {
          "200": {
            "content": {
              "application/json": {
                "schema": {
                  "$ref": "#/components/schemas/PowerCycleState"
                }
              }
            },
            "description": "Power Cycle State updated"
          },
          "401": {
            "$ref": "#/components/responses/Unauthorized"
          },
          "404": {
            "$ref": "#/components/responses/NotFound"
          },
          "500": {
            "$ref": "#/components/responses/InternalServerError"
          }
        },
        "security": [
          {
            "OAuth": [
              "oauth2"
            ]
          }
        ],
        "summary": "Update Power Cycle State",
        "tags": [
          "Power Cycle State"
        ]
      },
      "delete": {
        "responses": {
          "200": {
            "content": {
              "application/json": {
                "schema": {
                  "$ref": "#/components/schemas/PowerCycleState"
                }
              }
            },
            "description": "Power Cycle State restored to factory default"
          },
          "401": {
            "$ref": "#/components/responses/Unauthorized"
          },
          "404": {
            "$ref": "#/components/responses/NotFound"
          },
          "500": {
            "$ref": "#/components/responses/InternalServerError"
          }
        }
      },
    },
  },

```

```

    "security": [
      {
        "OAuth": [
          "oauth2"
        ]
      }
    ],
    "summary": "Set Power Cycle State to defaults",
    "description": "Deleting the Power Cycle State for a Device will cause it to revert\nto the o",
    "tags": [
      "Power Cycle State"
    ]
  },
  "parameters": [
    {
      "in": "path",
      "name": "device_id",
      "required": true,
      "schema": {
        "type": "number"
      }
    }
  ]
},
"/v2/devices/{device_id}/actions/{action_name}": {
  "put": {
    "requestBody": {
      "content": {
        "application/json": {
          "schema": {
            "$ref": "#/components/schemas/Args",
            "type": "object",
            "properties": {
              "_lock_priority": {
                "type": "number",
                "optional": true,
                "description": "The priority number desired for the lock, from 1 to 999.\nIf not",
              },
              "_lock_expiration": {
                "type": "number",
                "description": "The expiration time desired for the lock, in milliseconds. Use 0",
              }
            }
          },
          "example": {
            "argument": 4,
            "_lock_priority": 50,
            "_lock_expiration": 5000
          }
        }
      }
    }
  },
  "responses": {
    "200": {
      "description": "Action executed [since v2.12.4]",
      "content": {
        "application/json": {
          "schema": {
            "type": "object",
            "example": {
              "argument": 4,
              "_lock_priority": 50,
              "_lock_expiration": 4900
            }
          }
        }
      }
    },
    "204": {
      "description": "Action executed [prior to v2.12.4]"
    },
    "400": {
      "$ref": "#/components/responses/BadRequest"
    },
    "401": {
      "$ref": "#/components/responses/Unauthorized"
    }
  }
},

```



```

        "423": {
            "$ref": "#/components/responses/Locked"
        },
        "500": {
            "$ref": "#/components/responses/InternalServerError"
        }
    },
    "summary": "Execute a device action",
    "description": "`action_name` is the name of the action from the `actions` list in the response",
    "tags": [
        "Device Actions"
    ],
    "security": [
        {
            "OAuth": [
                "oauth2"
            ]
        }
    ]
},
"parameters": [
    {
        "in": "path",
        "name": "action_name",
        "required": true,
        "schema": {
            "type": "string"
        }
    },
    {
        "in": "path",
        "name": "device_id",
        "required": true,
        "schema": {
            "type": "number"
        }
    }
]
},
"/v2/devices/{device_id}/commands": {
    "parameters": [
        {
            "in": "path",
            "name": "device_id",
            "required": true,
            "schema": {
                "type": "string"
            }
        }
    ],
    "get": {
        "description": "Returns a list of command IDs and the corresponding command hashes.\nPlease see the response for more details.",
        "responses": {
            "200": {
                "content": {
                    "application/json": {
                        "schema": {
                            "$ref": "#/components/schemas/CommandList"
                        }
                    }
                },
                "description": "Command list returned"
            },
            "401": {
                "$ref": "#/components/responses/Unauthorized"
            },
            "404": {
                "$ref": "#/components/responses/NotFound"
            },
            "500": {
                "$ref": "#/components/responses/InternalServerError"
            }
        },
        "summary": "Get list of Commands",
        "tags": [
            "Device Commands"
        ]
    }
}

```

```

    },
    "post": {
      "requestBody": {
        "content": {
          "application/json": {
            "schema": {
              "$ref": "#/components/schemas/Command"
            }
          }
        }
      },
      "responses": {
        "201": {
          "$ref": "#/components/responses/Created"
        },
        "400": {
          "$ref": "#/components/responses/BadRequest"
        },
        "401": {
          "$ref": "#/components/responses/Unauthorized"
        },
        "500": {
          "$ref": "#/components/responses/InternalServerError"
        }
      },
      "summary": "Create new Command",
      "description": "Creates a Command under a specific Device.\n",
      "tags": [
        "Device Commands"
      ]
    },
    "/v2/devices/{device_id}/commands/{command_id}": {
      "parameters": [
        {
          "in": "path",
          "name": "device_id",
          "required": true,
          "schema": {
            "type": "string"
          }
        },
        {
          "in": "path",
          "name": "command_id",
          "required": true,
          "schema": {
            "type": "string"
          }
        }
      ],
      "get": {
        "responses": {
          "200": {
            "content": {
              "application/json": {
                "schema": {
                  "$ref": "#/components/schemas/Command"
                }
              }
            },
            "description": "Command object returned"
          },
          "401": {
            "$ref": "#/components/responses/Unauthorized"
          },
          "404": {
            "$ref": "#/components/responses/NotFound"
          },
          "500": {
            "$ref": "#/components/responses/InternalServerError"
          }
        },
        "summary": "Get specific Command",
        "description": "Get information about a Command.\n",
        "tags": [
          "Device Commands"
        ]
      }
    }
  }

```

```

    ]
  },
  "patch": {
    "requestBody": {
      "content": {
        "application/json": {
          "schema": {
            "$ref": "#/components/schemas/Command"
          }
        }
      }
    },
    "responses": {
      "200": {
        "description": "Command modified"
      },
      "400": {
        "$ref": "#/components/responses/BadRequest"
      },
      "401": {
        "$ref": "#/components/responses/Unauthorized"
      },
      "404": {
        "$ref": "#/components/responses/NotFound"
      },
      "500": {
        "$ref": "#/components/responses/InternalServerError"
      }
    },
    "summary": "Modify an existing Command",
    "description": "Modifies any fields of an existing Command.\n",
    "tags": [
      "Device Commands"
    ]
  },
  "delete": {
    "responses": {
      "204": {
        "description": "Command deleted"
      },
      "401": {
        "$ref": "#/components/responses/Unauthorized"
      },
      "404": {
        "$ref": "#/components/responses/NotFound"
      },
      "500": {
        "$ref": "#/components/responses/InternalServerError"
      }
    },
    "summary": "Delete a Command",
    "description": "Deletes specified Command.\n\nAny associated Signal will be implicitly deleted",
    "tags": [
      "Device Commands"
    ]
  }
},
"/v2/devices/{device_id}/commands/{command_id}/signal": {
  "parameters": [
    {
      "in": "path",
      "name": "device_id",
      "required": true,
      "schema": {
        "type": "string"
      }
    },
    {
      "in": "path",
      "name": "command_id",
      "required": true,
      "schema": {
        "type": "string"
      }
    }
  ],
  "get": {

```

```

"responses": {
  "200": {
    "description": "Signal returned",
    "content": {
      "application/json": {
        "schema": {
          "$ref": "#/components/schemas/Signal"
        }
      }
    }
  },
  "401": {
    "$ref": "#/components/responses/Unauthorized"
  },
  "404": {
    "$ref": "#/components/responses/NotFound"
  },
  "500": {
    "$ref": "#/components/responses/InternalServerError"
  }
},
"summary": "Get Signal associated with Command",
"description": "Get signal associated with this command.\n",
"tags": [
  "Device Command Signal"
]
},
"put": {
  "requestBody": {
    "content": {
      "application/json": {
        "schema": {
          "$ref": "#/components/schemas/Signal"
        }
      }
    }
  },
  "responses": {
    "200": {
      "description": "Signal set"
    },
    "400": {
      "$ref": "#/components/responses/BadRequest"
    },
    "401": {
      "$ref": "#/components/responses/Unauthorized"
    },
    "500": {
      "$ref": "#/components/responses/InternalServerError"
    }
  },
  "summary": "Store new Signal",
  "description": "Creates a Signal associated with this Command.\n",
  "tags": [
    "Device Command Signal"
  ]
},
"patch": {
  "requestBody": {
    "content": {
      "application/json": {
        "schema": {
          "$ref": "#/components/schemas/Signal"
        }
      }
    }
  },
  "responses": {
    "200": {
      "description": "Signal modified"
    },
    "400": {
      "$ref": "#/components/responses/BadRequest"
    },
    "401": {
      "$ref": "#/components/responses/Unauthorized"
    }
  },

```

```

        "404": {
            "$ref": "#/components/responses/NotFound"
        },
        "500": {
            "$ref": "#/components/responses/InternalServerError"
        }
    },
    "summary": "Modify an existing Signal",
    "description": "Modifies any fields of an existing Signal.\n",
    "tags": [
        "Device Command Signal"
    ]
},
"delete": {
    "responses": {
        "204": {
            "description": "Signal deleted"
        },
        "401": {
            "$ref": "#/components/responses/Unauthorized"
        },
        "404": {
            "$ref": "#/components/responses/NotFound"
        },
        "500": {
            "$ref": "#/components/responses/InternalServerError"
        }
    },
    "summary": "Delete a Signal",
    "description": "Deletes specified Signal.\n\nAttempts to use the corresponding Actions will fail",
    "tags": [
        "Device Command Signal"
    ]
}
},
"/v2/devices/{device_id}/commands/{command_id}/tx": {
    "parameters": [
        {
            "in": "path",
            "name": "device_id",
            "required": true,
            "schema": {
                "type": "string"
            }
        },
        {
            "in": "path",
            "name": "command_id",
            "required": true,
            "schema": {
                "type": "string"
            }
        }
    ],
    "put": {
        "requestBody": {
            "content": {
                "application/json": {
                    "example": {}
                }
            }
        },
        "responses": {
            "204": {
                "description": "Command transmitted"
            },
            "400": {
                "$ref": "#/components/responses/BadRequest"
            },
            "401": {
                "$ref": "#/components/responses/Unauthorized"
            },
            "500": {
                "$ref": "#/components/responses/InternalServerError"
            }
        },
        "summary": "Transmit Command",
    }
}

```

```

        "description": "Transmits the signal associated with the command.\n\nNote: This does NOT execute the command.\n\nNote: This does NOT execute the command.\n\nNote: This does NOT execute the command.",
        "tags": [
            "Device Command Transmit"
        ]
    },
    "/v2/devices/{device_id}/skeds": {
        "parameters": [
            {
                "in": "path",
                "name": "device_id",
                "required": true,
                "schema": {
                    "type": "string"
                }
            }
        ],
        "get": {
            "description": "Returns a list of sked_ids and the corresponding sked hashes.\nPlease see the Device Schedule list returned",
            "responses": {
                "200": {
                    "content": {
                        "application/json": {
                            "schema": {
                                "$ref": "#/components/schemas/SkedList"
                            }
                        }
                    },
                    "description": "Device Schedule list returned"
                },
                "401": {
                    "$ref": "#/components/responses/Unauthorized"
                },
                "404": {
                    "$ref": "#/components/responses/NotFound"
                },
                "500": {
                    "$ref": "#/components/responses/InternalServerError"
                }
            },
            "summary": "Get list of Device Schedules",
            "tags": [
                "Device Schedules"
            ]
        },
        "post": {
            "requestBody": {
                "content": {
                    "application/json": {
                        "schema": {
                            "$ref": "#/components/schemas/Sked"
                        }
                    }
                }
            },
            "responses": {
                "201": {
                    "$ref": "#/components/responses/Created"
                },
                "400": {
                    "$ref": "#/components/responses/BadRequest"
                },
                "401": {
                    "$ref": "#/components/responses/Unauthorized"
                },
                "500": {
                    "$ref": "#/components/responses/InternalServerError"
                }
            },
            "summary": "Create new Device Schedule",
            "description": "Creates a Device Schedule under a specific Device.\n\nIf the timezone or location is not specified, the default is the device's current timezone and location.",
            "tags": [
                "Device Schedules"
            ]
        }
    },
    "/v2/devices/{device_id}/skeds/{sked_id}": {

```

```

"parameters": [
  {
    "in": "path",
    "name": "device_id",
    "required": true,
    "schema": {
      "type": "string"
    }
  },
  {
    "in": "path",
    "name": "sked_id",
    "required": true,
    "schema": {
      "type": "string"
    }
  }
],
"get": {
  "responses": {
    "200": {
      "content": {
        "application/json": {
          "schema": {
            "$ref": "#/components/schemas/Sked"
          }
        }
      },
      "description": "Device Schedule object returned"
    },
    "401": {
      "$ref": "#/components/responses/Unauthorized"
    },
    "404": {
      "$ref": "#/components/responses/NotFound"
    },
    "500": {
      "$ref": "#/components/responses/InternalServerError"
    }
  },
  "summary": "Get specific Device Schedule",
  "description": "Get information about a Device Schedule.\n",
  "tags": [
    "Device Schedules"
  ]
},
"patch": {
  "requestBody": {
    "content": {
      "application/json": {
        "schema": {
          "$ref": "#/components/schemas/Sked"
        }
      }
    }
  },
  "responses": {
    "200": {
      "description": "Device Schedule modified"
    },
    "400": {
      "$ref": "#/components/responses/BadRequest"
    },
    "401": {
      "$ref": "#/components/responses/Unauthorized"
    },
    "404": {
      "$ref": "#/components/responses/NotFound"
    },
    "500": {
      "$ref": "#/components/responses/InternalServerError"
    }
  },
  "summary": "Modify an existing Device Schedule",
  "description": "Modifies any fields of an existing Device Schedule.\n",
  "tags": [
    "Device Schedules"
  ]
}

```

```

    ]
  },
  "delete": {
    "responses": {
      "204": {
        "description": "Device Schedule deleted"
      },
      "401": {
        "$ref": "#/components/responses/Unauthorized"
      },
      "404": {
        "$ref": "#/components/responses/NotFound"
      },
      "500": {
        "$ref": "#/components/responses/InternalServerError"
      }
    },
    "summary": "Delete a Device Schedule",
    "description": "Deletes specified Device Schedule.\n\nNaturally, the Device Schedule will be c",
    "tags": [
      "Device Schedules"
    ]
  }
},
"/v2/devices/{device_id}/vrc": {
  "get": {
    "description": "Check enabled status of virtual remote for specified device.\n",
    "responses": {
      "200": {
        "content": {
          "application/json": {
            "schema": {
              "$ref": "#/components/schemas/Vrc"
            }
          }
        }
      },
      "description": "Get Vrc info"
    },
    "401": {
      "$ref": "#/components/responses/Unauthorized"
    },
    "404": {
      "$ref": "#/components/responses/NotFound"
    },
    "500": {
      "$ref": "#/components/responses/InternalServerError"
    }
  },
  "security": [
    {
      "OAuth": [
        "oauth2"
      ]
    }
  ],
  "summary": "Get Vrc info",
  "tags": [
    "Virtual Remote Control"
  ]
},
"patch": {
  "requestBody": {
    "required": true,
    "content": {
      "application/json": {
        "schema": {
          "$ref": "#/components/schemas/Vrc"
        }
      }
    }
  },
  "responses": {
    "200": {
      "content": {
        "application/json": {
          "schema": {
            "$ref": "#/components/schemas/Vrc"
          }
        }
      }
    }
  }
}

```



```

    }
  },
  "description": "Enable or disable Vrc"
},
"401": {
  "$ref": "#/components/responses/Unauthorized"
},
"404": {
  "$ref": "#/components/responses/NotFound"
},
"500": {
  "$ref": "#/components/responses/InternalServerError"
}
},
"security": [
  {
    "BasicAuth": []
  }
],
"summary": "Enable or disable Vrc",
"tags": [
  "Virtual Remote Control"
]
},
"parameters": [
  {
    "in": "path",
    "name": "device_id",
    "required": true,
    "schema": {
      "type": "number"
    }
  }
],
"description": "testing 123"
},
"/v2/devices/{device_id}/vrc/keystream": {
  "put": {
    "requestBody": {
      "required": true,
      "content": {
        "application/json": {
          "schema": {
            "$ref": "#/components/schemas/VrcKeystream"
          }
        }
      }
    }
  },
  "responses": {
    "200": {
      "content": {
        "application/json": {
          "schema": {
            "$ref": "#/components/schemas/VrcKeystream"
          }
        }
      }
    },
    "description": "Send a virtual key event"
  },
  "401": {
    "$ref": "#/components/responses/Unauthorized"
  },
  "404": {
    "$ref": "#/components/responses/NotFound"
  },
  "500": {
    "$ref": "#/components/responses/InternalServerError"
  }
},
"security": [
  {
    "BasicAuth": []
  }
],
"summary": "Send a virtual key event",
"description": "Example:\n\nUser Behavior:\n - user taps 1\n - then holds 1\n - adds 2\n"

```

```

    "tags": [
      "Virtual Remote Control"
    ]
  },
  "parameters": [
    {
      "in": "path",
      "name": "device_id",
      "required": true,
      "schema": {
        "type": "number"
      }
    }
  ],
  "description": "testing 123"
},
"/v2/groups": {
  "get": {
    "description": "Returns a list of Group IDs and the corresponding Group hashes.\nThe hashes o",
    "responses": {
      "200": {
        "content": {
          "application/json": {
            "schema": {
              "$ref": "#/components/schemas/GroupList"
            }
          }
        },
        "description": "List Groups"
      },
      "401": {
        "$ref": "#/components/responses/Unauthorized"
      },
      "500": {
        "$ref": "#/components/responses/InternalServerError"
      }
    },
    "security": [
      {
        "OAuth": [
          "oauth2"
        ]
      }
    ],
    "summary": "List your Groups",
    "tags": [
      "Groups"
    ]
  },
  "post": {
    "requestBody": {
      "content": {
        "application/json": {
          "schema": {
            "$ref": "#/components/schemas/Group"
          }
        }
      }
    },
    "responses": {
      "201": {
        "$ref": "#/components/responses/Created"
      },
      "400": {
        "$ref": "#/components/responses/BadRequest"
      },
      "401": {
        "$ref": "#/components/responses/Unauthorized"
      },
      "409": {
        "$ref": "#/components/responses/Conflict"
      },
      "500": {
        "$ref": "#/components/responses/InternalServerError"
      }
    },
    "summary": "Create new Group",

```

```

        "description": "For the creation of a Group distributed across multiple Bonds,\nclients should use the following format",
        "tags": [
            "Groups"
        ]
    },
    "/v2/groups/{group_id}": {
        "get": {
            "description": "Groups are collections of Devices.\n",
            "responses": {
                "200": {
                    "content": {
                        "application/json": {
                            "schema": {
                                "$ref": "#/components/schemas/Group"
                            }
                        }
                    }
                },
                "description": "Get Group"
            },
            "401": {
                "$ref": "#/components/responses/Unauthorized"
            },
            "404": {
                "$ref": "#/components/responses/NotFound"
            },
            "500": {
                "$ref": "#/components/responses/InternalServerError"
            }
        },
        "security": [
            {
                "OAuth": [
                    "oauth2"
                ]
            }
        ],
        "summary": "Get Group",
        "tags": [
            "Groups"
        ]
    },
    "parameters": [
        {
            "in": "path",
            "name": "group_id",
            "required": true,
            "schema": {
                "type": "number"
            }
        }
    ],
    "patch": {
        "requestBody": {
            "required": true,
            "content": {
                "application/json": {
                    "schema": {
                        "$ref": "#/components/schemas/Group"
                    }
                }
            }
        },
        "responses": {
            "200": {
                "content": {
                    "application/json": {
                        "schema": {
                            "$ref": "#/components/schemas/Group"
                        }
                    }
                }
            },
            "description": "Change Group field"
        },
        "401": {
            "$ref": "#/components/responses/Unauthorized"
        }
    },

```

```

        "404": {
            "$ref": "#/components/responses/NotFound"
        },
        "500": {
            "$ref": "#/components/responses/InternalServerError"
        }
    },
    "security": [
        {
            "BasicAuth": []
        }
    ],
    "summary": "Change Group field",
    "description": "The `devices` list may be PATCHed to perform a bulk change to the membership",
    "tags": [
        "Groups"
    ]
},
"delete": {
    "responses": {
        "204": {
            "description": "Delete Group"
        },
        "401": {
            "$ref": "#/components/responses/Unauthorized"
        },
        "404": {
            "$ref": "#/components/responses/NotFound"
        },
        "500": {
            "$ref": "#/components/responses/InternalServerError"
        }
    },
    "security": [
        {
            "BasicAuth": []
        }
    ],
    "summary": "Delete Group",
    "description": "The entire Group is deleted.\n",
    "tags": [
        "Groups"
    ]
},
"/v2/groups/{group_id}/state": {
    "get": {
        "responses": {
            "200": {
                "content": {
                    "application/json": {
                        "schema": {
                            "$ref": "#/components/schemas/schemas_State"
                        }
                    }
                },
                "description": "Get Group State"
            },
            "401": {
                "$ref": "#/components/responses/Unauthorized"
            },
            "404": {
                "$ref": "#/components/responses/NotFound"
            },
            "500": {
                "$ref": "#/components/responses/InternalServerError"
            }
        },
        "security": [
            {
                "OAuth": [
                    "oauth2"
                ]
            }
        ],
        "summary": "Get Group State",
        "description": "Group State lists those state variables which are common to all member\nDevi

```

```

    "tags": [
      "Group State"
    ]
  },
  "parameters": [
    {
      "in": "path",
      "name": "group_id",
      "required": true,
      "schema": {
        "type": "number"
      }
    }
  ]
},
"/v2/groups/{group_id}/properties": {
  "get": {
    "responses": {
      "200": {
        "content": {
          "application/json": {
            "schema": {
              "$ref": "#/components/schemas/Properties"
            }
          }
        }
      },
      "description": "Get Group properties"
    },
    "401": {
      "$ref": "#/components/responses/Unauthorized"
    },
    "404": {
      "$ref": "#/components/responses/NotFound"
    },
    "500": {
      "$ref": "#/components/responses/InternalServerError"
    }
  },
  "security": [
    {
      "OAuth": [
        "oauth2"
      ]
    }
  ],
  "summary": "Get Group properties",
  "description": "See the Features section above for other properties which may be available\n",
  "tags": [
    "Group Properties"
  ]
},
"patch": {
  "requestBody": {
    "content": {
      "application/json": {
        "schema": {
          "$ref": "#/components/schemas/Properties"
        }
      }
    }
  },
  "responses": {
    "200": {
      "description": "Properties updated"
    },
    "401": {
      "$ref": "#/components/responses/Unauthorized"
    },
    "404": {
      "$ref": "#/components/responses/NotFound"
    },
    "500": {
      "$ref": "#/components/responses/InternalServerError"
    }
  },
  "security": [
    {

```

```

        "OAuth": [
            "oauth2"
        ]
    },
    "summary": "Update properties",
    "description": "PATCHing one or more Group Properties implicitly PATCHes those Properties\n",
    "tags": [
        "Group Properties"
    ]
},
"parameters": [
    {
        "in": "path",
        "name": "group_id",
        "required": true,
        "schema": {
            "type": "number"
        }
    }
]
},
"/v2/groups/{group_id}/actions/{action_name}": {
    "put": {
        "requestBody": {
            "content": {
                "application/json": {
                    "schema": {
                        "type": "object",
                        "example": {
                            argument: 4
                        }
                    }
                }
            }
        },
        "responses": {
            "200": {
                "description": "Action executed",
                "content": {
                    "application/json": {
                        "schema": {
                            "type": "object",
                            "example": {
                                argument: 4
                            }
                        }
                    }
                }
            },
            "400": {
                "$ref": "#/components/responses/BadRequest"
            },
            "401": {
                "$ref": "#/components/responses/Unauthorized"
            },
            "500": {
                "$ref": "#/components/responses/InternalServerError"
            }
        },
        "summary": "Execute a Group Action",
        "description": "`action_name` is the name of the action from the `actions` list in\nthe response",
        "tags": [
            "Group Actions"
        ]
    },
    "security": [
        {
            "OAuth": [
                "oauth2"
            ]
        }
    ]
},
"parameters": [
    {
        "in": "path",
        "name": "action_name",

```

```

        "required": true,
        "schema": {
            "type": "string"
        }
    },
    {
        "in": "path",
        "name": "group_id",
        "required": true,
        "schema": {
            "type": "number"
        }
    }
]
},
"/v2/groups/{group_id}/skeds": {
    "parameters": [
        {
            "in": "path",
            "name": "group_id",
            "required": true,
            "schema": {
                "type": "string"
            }
        }
    ],
    "get": {
        "description": "Returns a list of sked_ids and the corresponding sked hashes.\nPlease see the",
        "responses": {
            "200": {
                "content": {
                    "application/json": {
                        "schema": {
                            "$ref": "#/components/schemas/SkedList"
                        }
                    }
                },
                "description": "Group Schedule list returned"
            },
            "401": {
                "$ref": "#/components/responses/Unauthorized"
            },
            "404": {
                "$ref": "#/components/responses/NotFound"
            },
            "500": {
                "$ref": "#/components/responses/InternalServerError"
            }
        },
        "summary": "Get list of Group Schedules",
        "tags": [
            "Group Schedules"
        ]
    },
    "post": {
        "requestBody": {
            "content": {
                "application/json": {
                    "schema": {
                        "$ref": "#/components/schemas/Sked"
                    }
                }
            }
        },
        "responses": {
            "201": {
                "$ref": "#/components/responses/Created"
            },
            "400": {
                "$ref": "#/components/responses/BadRequest"
            },
            "401": {
                "$ref": "#/components/responses/Unauthorized"
            },
            "500": {
                "$ref": "#/components/responses/InternalServerError"
            }
        }
    }
}

```

```

    },
    "summary": "Create new Group Schedule",
    "description": "Creates a Group Schedule under a specific Group.\n\nIf the timezone or locat.",
    "tags": [
        "Group Schedules"
    ]
}
},
"/v2/groups/{group_id}/skeds/{sked_id}": {
    "parameters": [
        {
            "in": "path",
            "name": "group_id",
            "required": true,
            "schema": {
                "type": "string"
            }
        },
        {
            "in": "path",
            "name": "sked_id",
            "required": true,
            "schema": {
                "type": "string"
            }
        }
    ],
    "get": {
        "responses": {
            "200": {
                "content": {
                    "application/json": {
                        "schema": {
                            "$ref": "#/components/schemas/Sked"
                        }
                    }
                },
                "description": "Group Schedule object returned"
            },
            "401": {
                "$ref": "#/components/responses/Unauthorized"
            },
            "404": {
                "$ref": "#/components/responses/NotFound"
            },
            "500": {
                "$ref": "#/components/responses/InternalServerError"
            }
        },
        "summary": "Get specific Group Schedule",
        "description": "Get information about a Group Schedule.\n",
        "tags": [
            "Group Schedules"
        ]
    },
    "patch": {
        "requestBody": {
            "content": {
                "application/json": {
                    "schema": {
                        "$ref": "#/components/schemas/Sked"
                    }
                }
            }
        },
        "responses": {
            "200": {
                "description": "Group Schedule modified"
            },
            "400": {
                "$ref": "#/components/responses/BadRequest"
            },
            "401": {
                "$ref": "#/components/responses/Unauthorized"
            },
            "404": {
                "$ref": "#/components/responses/NotFound"
            }
        }
    }
}

```



```

    },
    "500": {
      "$ref": "#/components/responses/InternalServerError"
    }
  },
  "summary": "Modify an existing Group Schedule",
  "description": "Modifies any fields of an existing Group Schedule.\n",
  "tags": [
    "Group Schedules"
  ]
},
"delete": {
  "responses": {
    "204": {
      "description": "Group Schedule deleted"
    },
    "401": {
      "$ref": "#/components/responses/Unauthorized"
    },
    "404": {
      "$ref": "#/components/responses/NotFound"
    },
    "500": {
      "$ref": "#/components/responses/InternalServerError"
    }
  },
  "summary": "Delete a Group Schedule",
  "description": "Deletes specified Group Schedule.\nNaturally, the Group Schedule will be cance\n",
  "tags": [
    "Group Schedules"
  ]
}
},
"/v2/scenes": {
  "get": {
    "description": "Returns a list of Scene IDs and the corresponding Scene hashes.\n\nThe hashes change if any part of the Scene (its name, state, type, etc.)\nhas changed. The hash is not updated if underlying Actors (Devices and/or Groups) are updated\nin a way which does not effect the Scene state or type.\n\n\nScenes are supported on Bridges and on Smart by Bond.\nScenes may span multiple Bonds (Bridges and SBB devices) by using the same\nScene ID on each Bond. See the introductory section \"Groups and Scenes\".\n",
    "responses": {
      "200": {
        "content": {
          "application/json": {
            "schema": {
              "$ref": "#/components/schemas/SceneList"
            }
          }
        },
        "description": "List Scenes"
      },
      "401": {
        "$ref": "#/components/responses/Unauthorized"
      },
      "500": {
        "$ref": "#/components/responses/InternalServerError"
      }
    },
    "security": [
      {
        "OAuth": [
          "oauth2"
        ]
      }
    ],
    "summary": "List your Scenes",
    "tags": [
      "Scenes"
    ]
  },
  "post": {
    "requestBody": {
      "content": {
        "application/json": {

```

```

        "schema": {
          "$ref": "#/components/schemas/Scene"
        }
      },
    },
    "responses": {
      "201": {
        "$ref": "#/components/responses/Created"
      },
      "400": {
        "$ref": "#/components/responses/BadRequest"
      },
      "401": {
        "$ref": "#/components/responses/Unauthorized"
      },
      "409": {
        "$ref": "#/components/responses/Conflict"
      },
      "500": {
        "$ref": "#/components/responses/InternalServerError"
      }
    },
    "summary": "Create new Scene",
    "description": "For the creation of a Scene distributed across multiple Bonds,\nclients should",
    "tags": [
      "Scenes"
    ]
  },
  "/v2/scenes/{scene_id}": {
    "get": {
      "description": "Scenes are a set of actions to run on Devices and/or Groups\nwith a single r",
      "responses": {
        "200": {
          "content": {
            "application/json": {
              "schema": {
                "$ref": "#/components/schemas/Scene"
              }
            }
          }
        },
        "401": {
          "$ref": "#/components/responses/Unauthorized"
        },
        "404": {
          "$ref": "#/components/responses/NotFound"
        },
        "500": {
          "$ref": "#/components/responses/InternalServerError"
        }
      },
      "security": [
        {
          "OAuth": [
            "oauth2"
          ]
        }
      ],
      "summary": "Get Scene",
      "tags": [
        "Scenes"
      ]
    },
    "parameters": [
      {
        "in": "path",
        "name": "scene_id",
        "required": true,
        "schema": {
          "type": "number"
        }
      }
    ],
    "patch": {

```

```

    "requestBody": {
      "content": {
        "application/json": {
          "schema": {
            "$ref": "#/components/schemas/Scene"
          }
        }
      }
    },
    "responses": {
      "200": {
        "content": {
          "application/json": {
            "schema": {
              "$ref": "#/components/schemas/Scene"
            }
          }
        },
        "description": "Change Scene field"
      },
      "401": {
        "$ref": "#/components/responses/Unauthorized"
      },
      "404": {
        "$ref": "#/components/responses/NotFound"
      },
      "500": {
        "$ref": "#/components/responses/InternalServerError"
      }
    },
    "security": [
      {
        "BasicAuth": []
      }
    ],
    "summary": "Change Scene field",
    "description": "The `actors` list may be PATCHed to perform a bulk change to the membership.",
    "tags": [
      "Scenes"
    ]
  },
  "delete": {
    "responses": {
      "204": {
        "description": "Delete Scene"
      },
      "401": {
        "$ref": "#/components/responses/Unauthorized"
      },
      "404": {
        "$ref": "#/components/responses/NotFound"
      },
      "500": {
        "$ref": "#/components/responses/InternalServerError"
      }
    },
    "security": [
      {
        "BasicAuth": []
      }
    ],
    "summary": "Delete Scene",
    "description": "The entire Scene is deleted.\n",
    "tags": [
      "Scenes"
    ]
  }
},
"/v2/scenes/{scene_id}/run": {
  "put": {
    "description": "Execute specified Scene.\nEach actor will be executed individually.\n\nFor s",
    "responses": {
      "204": {
        "description": "Scene executed"
      },
      "401": {
        "$ref": "#/components/responses/Unauthorized"
      }
    }
  }
}

```

```

    },
    "404": {
      "$ref": "#/components/responses/NotFound"
    },
    "500": {
      "$ref": "#/components/responses/InternalServerError"
    }
  },
  "security": [
    {
      "OAuth": [
        "oauth2"
      ]
    }
  ],
  "summary": "Execute Scene",
  "tags": [
    "Scenes"
  ]
},
"parameters": [
  {
    "in": "path",
    "name": "scene_id",
    "required": true,
    "schema": {
      "type": "number"
    }
  }
]
},
"/v2/scenes/{scene_id}/skeds": {
  "parameters": [
    {
      "in": "path",
      "name": "scene_id",
      "required": true,
      "schema": {
        "type": "string"
      }
    }
  ],
  "get": {
    "description": "Returns a list of sked_ids and the corresponding sked hashes.\nPlease see the",
    "responses": {
      "200": {
        "content": {
          "application/json": {
            "schema": {
              "$ref": "#/components/schemas/SkedList"
            }
          }
        },
        "description": "Scene Schedule list returned"
      },
      "401": {
        "$ref": "#/components/responses/Unauthorized"
      },
      "404": {
        "$ref": "#/components/responses/NotFound"
      },
      "500": {
        "$ref": "#/components/responses/InternalServerError"
      }
    },
    "summary": "Get list of Scene Schedules",
    "tags": [
      "Scene Schedules"
    ]
  },
  "post": {
    "requestBody": {
      "content": {
        "application/json": {
          "schema": {
            "$ref": "#/components/schemas/Sked"
          }
        }
      }
    }
  }
}

```

```

    }
  },
  "responses": {
    "201": {
      "$ref": "#/components/responses/Created"
    },
    "400": {
      "$ref": "#/components/responses/BadRequest"
    },
    "401": {
      "$ref": "#/components/responses/Unauthorized"
    },
    "500": {
      "$ref": "#/components/responses/InternalServerError"
    }
  },
  "summary": "Create new Scene Schedule",
  "description": "Creates a Scene Schedule under a specific Scene.\n\nNote that `action` must be one of the following values: 'create', 'update', 'delete'.",
  "tags": [
    "Scene Schedules"
  ]
},
"/v2/scenes/{scene_id}/skeds/{sked_id}": {
  "parameters": [
    {
      "in": "path",
      "name": "scene_id",
      "required": true,
      "schema": {
        "type": "string"
      }
    },
    {
      "in": "path",
      "name": "sked_id",
      "required": true,
      "schema": {
        "type": "string"
      }
    }
  ],
  "get": {
    "responses": {
      "200": {
        "content": {
          "application/json": {
            "schema": {
              "$ref": "#/components/schemas/Sked"
            }
          }
        },
        "description": "Scene Schedule object returned"
      },
      "401": {
        "$ref": "#/components/responses/Unauthorized"
      },
      "404": {
        "$ref": "#/components/responses/NotFound"
      },
      "500": {
        "$ref": "#/components/responses/InternalServerError"
      }
    },
    "summary": "Get specific Scene Schedule",
    "description": "Get information about a Scene Schedule.\n",
    "tags": [
      "Scene Schedules"
    ]
  },
  "patch": {
    "requestBody": {
      "content": {
        "application/json": {
          "schema": {
            "$ref": "#/components/schemas/Sked"
          }
        }
      }
    }
  }
}

```

```

    }
  }
},
"responses": {
  "200": {
    "description": "Scene Schedule modified"
  },
  "400": {
    "$ref": "#/components/responses/BadRequest"
  },
  "401": {
    "$ref": "#/components/responses/Unauthorized"
  },
  "404": {
    "$ref": "#/components/responses/NotFound"
  },
  "500": {
    "$ref": "#/components/responses/InternalServerError"
  }
},
"summary": "Modify an existing Scene Schedule",
"description": "Modifies any fields of an existing Scene Schedule.\n",
"tags": [
  "Scene Schedules"
]
},
"delete": {
  "responses": {
    "204": {
      "description": "Scene Schedule deleted"
    },
    "401": {
      "$ref": "#/components/responses/Unauthorized"
    },
    "404": {
      "$ref": "#/components/responses/NotFound"
    },
    "500": {
      "$ref": "#/components/responses/InternalServerError"
    }
  },
  "summary": "Delete a Scene Schedule",
  "description": "Deletes specified Scene Schedule.\nNaturally, the Scene Schedule will be canceled",
  "tags": [
    "Scene Schedules"
  ]
}
},
"/v2/ports": {
  "get": {
    "description": "Returns a list of physical port IDs and the corresponding port hashes.\nThe list of ports is returned in the response body.",
    "responses": {
      "200": {
        "content": {
          "application/json": {
            "schema": {
              "$ref": "#/components/schemas/PortList"
            }
          }
        },
        "description": "Port list returned"
      },
      "401": {
        "$ref": "#/components/responses/Unauthorized"
      },
      "500": {
        "$ref": "#/components/responses/InternalServerError"
      }
    },
    "summary": "List physical ports",
    "tags": [
      "Ports"
    ]
  }
},
"/v2/ports/_meta": {

```

```

    "get": {
      "description": "Returns the enum values supported by this firmware build for the\nwritable f",
      "responses": {
        "200": {
          "content": {
            "application/json": {
              "schema": {
                "$ref": "#/components/schemas/PortsMeta"
              }
            }
          },
          "description": "Ports meta document returned"
        },
        "401": {
          "$ref": "#/components/responses/Unauthorized"
        },
        "500": {
          "$ref": "#/components/responses/InternalServerError"
        }
      },
      "summary": "Get supported port enum values",
      "tags": [
        "Ports"
      ]
    },
    "/v2/ports/{port_id}": {
      "parameters": [
        {
          "in": "path",
          "name": "port_id",
          "required": true,
          "schema": {
            "type": "string"
          }
        }
      ],
      "get": {
        "responses": {
          "200": {
            "content": {
              "application/json": {
                "schema": {
                  "$ref": "#/components/schemas/Port"
                }
              }
            },
            "description": "Port object returned"
          },
          "401": {
            "$ref": "#/components/responses/Unauthorized"
          },
          "404": {
            "$ref": "#/components/responses/NotFound"
          },
          "500": {
            "$ref": "#/components/responses/InternalServerError"
          }
        },
        "summary": "Get a port's configuration",
        "description": "Get the current configuration of a physical port. Only fields present\nin the",
        "tags": [
          "Ports"
        ]
      },
      "patch": {
        "requestBody": {
          "content": {
            "application/json": {
              "schema": {
                "$ref": "#/components/schemas/Port"
              }
            }
          }
        },
        "responses": {
          "200": {

```

```

        "content": {
          "application/json": {
            "schema": {
              "$ref": "#/components/schemas/Port"
            }
          }
        },
        "description": "Port modified – returns the updated port object."
      },
      "400": {
        "$ref": "#/components/responses/BadRequest"
      },
      "401": {
        "$ref": "#/components/responses/Unauthorized"
      },
      "404": {
        "$ref": "#/components/responses/NotFound"
      },
      "500": {
        "$ref": "#/components/responses/InternalServerError"
      }
    },
    "summary": "Modify a port's configuration",
    "description": "Modify writable fields of a port. Read-only fields (`port_type`,\n`port_class`)",
    "tags": [
      "Ports"
    ]
  }
},
"/v2/sidekicks": {
  "get": {
    "description": "Returns a list of Sidekicks.\n",
    "responses": {
      "200": {
        "content": {
          "application/json": {
            "schema": {
              "$ref": "#/components/schemas/SidekickList"
            }
          }
        },
        "description": "Sidekick list returned"
      },
      "401": {
        "$ref": "#/components/responses/Unauthorized"
      },
      "404": {
        "$ref": "#/components/responses/NotFound"
      },
      "500": {
        "$ref": "#/components/responses/InternalServerError"
      }
    },
    "summary": "Get list of Sidekicks",
    "tags": [
      "Sidekicks"
    ]
  },
  "post": {
    "requestBody": {
      "content": {
        "application/json": {
          "schema": {
            "$ref": "#/components/schemas/Sidekick"
          }
        }
      }
    },
    "responses": {
      "201": {
        "$ref": "#/components/responses/Created"
      },
      "400": {
        "$ref": "#/components/responses/BadRequest"
      },
      "401": {
        "$ref": "#/components/responses/Unauthorized"
      }
    }
  }
}

```



```

    },
    "500": {
      "$ref": "#/components/responses/InternalServerError"
    }
  },
  "summary": "Add a Sidekick to Bridge.",
  "description": "Adds a Sidekick by ID. The Sidekick ID shall be specified by client\nin the",
  "tags": [
    "Sidekick"
  ]
}
},
"/v2/sidekicks/{sk_id}": {
  "parameters": [
    {
      "in": "path",
      "name": "sk_id",
      "required": true,
      "schema": {
        "type": "string"
      }
    }
  ],
  "get": {
    "responses": {
      "200": {
        "content": {
          "application/json": {
            "schema": {
              "$ref": "#/components/schemas/Sidekick"
            }
          }
        },
        "description": "Sidekick object returned"
      },
      "401": {
        "$ref": "#/components/responses/Unauthorized"
      },
      "404": {
        "$ref": "#/components/responses/NotFound"
      },
      "500": {
        "$ref": "#/components/responses/InternalServerError"
      }
    },
    "summary": "Get specific Sidekick",
    "description": "Get information about a Sidekick.\n",
    "tags": [
      "Sidekick"
    ]
  },
  "patch": {
    "requestBody": {
      "content": {
        "application/json": {
          "schema": {
            "$ref": "#/components/schemas/Sidekick"
          }
        }
      }
    },
    "responses": {
      "200": {
        "description": "Sidekick modified"
      },
      "400": {
        "$ref": "#/components/responses/BadRequest"
      },
      "401": {
        "$ref": "#/components/responses/Unauthorized"
      },
      "404": {
        "$ref": "#/components/responses/NotFound"
      },
      "500": {
        "$ref": "#/components/responses/InternalServerError"
      }
    }
  }
}

```

```

    },
    "summary": "Modify an existing Sidekick's metadata and links",
    "description": "Modifies metadata and/or links of a Sidekick.\n\nThe `chan_link` field has comp",
    "tags": [
      "Sidekick"
    ]
  },
  "delete": {
    "responses": {
      "204": {
        "description": "Sidekick deleted"
      },
      "401": {
        "$ref": "#/components/responses/Unauthorized"
      },
      "404": {
        "$ref": "#/components/responses/NotFound"
      },
      "500": {
        "$ref": "#/components/responses/InternalServerError"
      }
    },
    "summary": "Delete a Sidekick",
    "description": "Deletes specified Sidekick.\n",
    "tags": [
      "Sidekick"
    ]
  }
},
"/v2/sidekicks/{sk_id}/keystream": {
  "parameters": [
    {
      "in": "path",
      "name": "sk_id",
      "required": true,
      "schema": {
        "type": "string"
      }
    }
  ],
  "get": {
    "responses": {
      "200": {
        "content": {
          "application/json": {
            "schema": {
              "$ref": "#/components/schemas/Keystream"
            }
          }
        },
        "description": "Keystream object"
      },
      "204": {
        "description": "No content, only available via BPUP."
      },
      "401": {
        "$ref": "#/components/responses/Unauthorized"
      },
      "404": {
        "$ref": "#/components/responses/NotFound"
      },
      "500": {
        "$ref": "#/components/responses/InternalServerError"
      }
    },
    "summary": "Get key event.",
    "description": "This endpoint is pushed whenever a new key event occurs on the \n\nCorresponding",
    "tags": [
      "Keystream"
    ]
  }
},
"/v2/sidekicks/_learn": {
  "get": {
    "responses": {
      "200": {
        "content": {

```

```

        "application/json": {
            "schema": {
                "$ref": "#/components/schemas/Learn"
            }
        },
        "description": "Get Sidekick learn status"
    },
    "401": {
        "$ref": "#/components/responses/Unauthorized"
    },
    "404": {
        "$ref": "#/components/responses/NotFound"
    },
    "500": {
        "$ref": "#/components/responses/InternalServerError"
    }
},
"summary": "Get Sidekick learn status",
"description": "[BBP-only]\n\nProvides information about the Sidekick learn window on this B",
"tags": [
    "Sidekick Learn"
]
},
"patch": {
    "requestBody": {
        "content": {
            "application/json": {
                "schema": {
                    "$ref": "#/components/schemas/Learn"
                }
            }
        }
    },
    "responses": {
        "200": {
            "content": {
                "application/json": {
                    "schema": {
                        "$ref": "#/components/schemas/Learn"
                    }
                }
            },
            "description": "Open or close Sidekick learn window"
        },
        "401": {
            "$ref": "#/components/responses/Unauthorized"
        },
        "404": {
            "$ref": "#/components/responses/NotFound"
        },
        "500": {
            "$ref": "#/components/responses/InternalServerError"
        }
    },
    "summary": "Open or close Sidekick learn window",
    "description": "[BBP-only]\n\nOpen/close the learn window.\n\nThe learn window is never automa",
    "tags": [
        "Sidekick Learn"
    ]
},
"delete": {
    "responses": {
        "200": {
            "description": "Learn window closed.",
            "content": {
                "application/json": {
                    "schema": {
                        "$ref": "#/components/schemas/Learn"
                    }
                }
            }
        },
        "401": {
            "$ref": "#/components/responses/Unauthorized"
        },
        "404": {

```

```

        "$ref": "#/components/responses/NotFound"
      },
      "500": {
        "$ref": "#/components/responses/InternalServerError"
      }
    },
    "summary": "Close learn window.",
    "description": "Close the learn window. Identical to PATCH learn_window_open=false.\n",
    "tags": [
      "Sidekick Learn"
    ]
  },
  "parameters": [
    {
      "in": "path",
      "name": "sk_id",
      "required": true,
      "schema": {
        "type": "string"
      }
    }
  ]
},
"/v2/signal/tx": {
  "delete": {
    "responses": {
      "204": {
        "description": "Signal transmission canceled"
      },
      "400": {
        "$ref": "#/components/responses/BadRequest"
      },
      "401": {
        "$ref": "#/components/responses/Unauthorized"
      },
      "404": {
        "$ref": "#/components/responses/NotFound"
      },
      "500": {
        "$ref": "#/components/responses/InternalServerError"
      }
    },
    "summary": "Cancel a Transmission",
    "description": "If a Signal is being transmitted, it is immediately interrupted,\nand a 204 .",
    "tags": [
      "Transmit"
    ]
  },
  "put": {
    "requestBody": {
      "content": {
        "application/json": {
          "schema": {
            "$ref": "#/components/schemas/Signal"
          }
        }
      }
    }
  },
  "responses": {
    "204": {
      "description": "Signal transmit started"
    },
    "400": {
      "$ref": "#/components/responses/BadRequest"
    },
    "401": {
      "$ref": "#/components/responses/Unauthorized"
    },
    "500": {
      "$ref": "#/components/responses/InternalServerError"
    }
  },
  "summary": "Transmit a Signal",
  "description": "Transmit an RF or IR signal from a Bond Bridge.\n\nIf `use_scan` is present a",
  "tags": [
    "Transmit"
  ]
}

```

```

    }
  },
  "/v2/signal/rssi": {
    "get": {
      "responses": {
        "200": {
          "description": "Receive Signal Strength",
          "content": {
            "application/json": {
              "schema": {
                "$ref": "#/components/schemas/Rssi"
              }
            }
          }
        },
        "401": {
          "$ref": "#/components/responses/Unauthorized"
        },
        "404": {
          "$ref": "#/components/responses/NotFound"
        },
        "500": {
          "$ref": "#/components/responses/InternalServerError"
        }
      },
      "summary": "Get Receive Signal Strength Indication (RSSI)",
      "description": "Sweeps over the frequency range supported by the product,\nand returns a list of RSSI values.",
      "tags": [
        "Rssi"
      ]
    }
  },
  "/v2/signal/scan": {
    "put": {
      "requestBody": {
        "content": {
          "application/json": {
            "schema": {
              "$ref": "#/components/schemas/Scan"
            }
          }
        }
      },
      "responses": {
        "201": {
          "description": "Signal transmit started"
        },
        "400": {
          "$ref": "#/components/responses/BadRequest"
        },
        "401": {
          "$ref": "#/components/responses/Unauthorized"
        },
        "500": {
          "$ref": "#/components/responses/InternalServerError"
        }
      },
      "summary": "Start a Scan",
      "description": "Start scanning for RF/IR signals.\n",
      "tags": [
        "Scan"
      ]
    },
    "get": {
      "responses": {
        "200": {
          "description": "Scan info returned",
          "content": {
            "application/json": {
              "schema": {
                "$ref": "#/components/schemas/Scan"
              }
            }
          }
        },
        "401": {
          "$ref": "#/components/responses/Unauthorized"
        }
      }
    }
  }
}

```

```

    },
    "404": {
      "$ref": "#/components/responses/NotFound"
    },
    "500": {
      "$ref": "#/components/responses/InternalServerError"
    }
  },
  "summary": "Check scan progress",
  "description": "Get information about a scan in progress,\nor the last completed scan.\n",
  "tags": [
    "Scan"
  ]
},
"delete": {
  "responses": {
    "204": {
      "description": "Scan canceled"
    },
    "401": {
      "$ref": "#/components/responses/Unauthorized"
    },
    "404": {
      "$ref": "#/components/responses/NotFound"
    },
    "500": {
      "$ref": "#/components/responses/InternalServerError"
    }
  },
  "summary": "Cancel a Scan",
  "description": "Immediately stop scanning, and delete results.\n",
  "tags": [
    "Scan"
  ]
}
},
"/v2/signal/scan/signal": {
  "get": {
    "responses": {
      "200": {
        "description": "Signal resulting from scan",
        "content": {
          "application/json": {
            "schema": {
              "$ref": "#/components/schemas/Signal"
            }
          }
        }
      },
      "401": {
        "$ref": "#/components/responses/Unauthorized"
      },
      "404": {
        "$ref": "#/components/responses/NotFound"
      },
      "500": {
        "$ref": "#/components/responses/InternalServerError"
      }
    },
    "summary": "Get Signal from a Scan",
    "description": "Get the signal resulting from the most recent scan\n",
    "tags": [
      "Scan Result"
    ]
  }
},
"/v2/token": {
  "get": {
    "responses": {
      "200": {
        "content": {
          "application/json": {
            "schema": {
              "$ref": "#/components/schemas/TokenGet"
            }
          }
        }
      }
    }
  }
},

```

```

        "description": "Local token status"
      },
      "401": {
        "$ref": "#/components/responses/Unauthorized"
      }
    },
    "summary": "Get Local API Token",
    "description": "Request token for accessing Bond over local network.\n\nIf `locked` is true,
    "tags": [
      "Token"
    ]
  },
  "patch": {
    "requestBody": {
      "content": {
        "application/json": {
          "schema": {
            "$ref": "#/components/schemas/TokenPatch"
          }
        }
      }
    },
    "responses": {
      "200": {
        "content": {
          "application/json": {
            "schema": {
              "$ref": "#/components/schemas/TokenPatch"
            }
          }
        },
        "description": "Token unlocked"
      },
      "400": {
        "$ref": "#/components/responses/BadRequest"
      },
      "401": {
        "$ref": "#/components/responses/Unauthorized"
      },
      "500": {
        "$ref": "#/components/responses/InternalServerError"
      }
    },
    "summary": "Add Bond to an Account (or Unlock Token)",
    "description": "If locked, provide `pin` (located on the product label) to unlock this endpoint.",
    "tags": [
      "Token"
    ]
  }
},
"/v2/api/mqtt": {
  "patch": {
    "requestBody": {
      "content": {
        "application/json": {
          "schema": {
            "$ref": "#/components/schemas/mqtt_Schema"
          }
        }
      }
    },
    "responses": {
      "200": {
        "description": "MQTT Reconfigured"
      },
      "401": {
        "$ref": "#/components/responses/Unauthorized"
      },
      "500": {
        "$ref": "#/components/responses/InternalServerError"
      }
    },
    "security": [
      {
        "BasicAuth": []
      }
    ]
  },

```

```

        "summary": "Configure MQTT Transport",
        "description": "IMPORTANT NOTE: MQTT redirection is currently untested. If you have a use\ncase",
        "tags": [
            "MQTT"
        ]
    },
    "get": {
        "responses": {
            "200": {
                "description": "MQTT Configuration",
                "content": {
                    "application/json": {
                        "schema": {
                            "$ref": "#/components/schemas/mqtt_Schema"
                        }
                    }
                }
            },
            "204": {
                "description": "No custom MQTT configuration found"
            },
            "401": {
                "$ref": "#/components/responses/Unauthorized"
            },
            "500": {
                "$ref": "#/components/responses/InternalServerError"
            }
        },
        "security": [
            {
                "BasicAuth": []
            }
        ],
        "summary": "Get MQTT Configuration",
        "tags": [
            "MQTT"
        ]
    },
    "delete": {
        "responses": {
            "204": {
                "description": "MQTT config reset to default"
            },
            "401": {
                "$ref": "#/components/responses/Unauthorized"
            },
            "500": {
                "$ref": "#/components/responses/InternalServerError"
            }
        },
        "security": [
            {
                "BasicAuth": []
            }
        ],
        "summary": "Reset MQTT Config to defaults",
        "tags": [
            "MQTT"
        ]
    }
},
"/v2/api/bpup": {
    "patch": {
        "requestBody": {
            "content": {
                "application/json": {
                    "schema": {
                        "$ref": "#/components/schemas/bpup_Schema"
                    }
                }
            }
        },
        "responses": {
            "200": {
                "description": "BPUP Reconfigured"
            },
            "401": {

```



```

        "$ref": "#/components/responses/Unauthorized"
      },
      "500": {
        "$ref": "#/components/responses/InternalServerError"
      }
    },
    "security": [
      {
        "BasicAuth": []
      }
    ],
    "summary": "Configure BPUP Transport",
    "description": "Configure the Bond's BPUP transport.\n",
    "tags": [
      "BPUP"
    ]
  },
  "get": {
    "responses": {
      "200": {
        "description": "BPUP Configuration",
        "content": {
          "application/json": {
            "schema": {
              "$ref": "#/components/schemas/bpup_Schema"
            }
          }
        }
      },
      "401": {
        "$ref": "#/components/responses/Unauthorized"
      },
      "500": {
        "$ref": "#/components/responses/InternalServerError"
      }
    },
    "security": [
      {
        "BasicAuth": []
      }
    ],
    "summary": "Get BPUP Configuration",
    "tags": [
      "BPUP"
    ]
  },
  "delete": {
    "responses": {
      "204": {
        "description": "BPUP config reset to default"
      },
      "401": {
        "$ref": "#/components/responses/Unauthorized"
      },
      "500": {
        "$ref": "#/components/responses/InternalServerError"
      }
    },
    "security": [
      {
        "BasicAuth": []
      }
    ],
    "summary": "Reset BPUP Config to defaults",
    "tags": [
      "BPUP"
    ]
  }
},
"/v2/api/bhk": {
  "get": {
    "responses": {
      "200": {
        "description": "Bond HomeKit configuration.",
        "content": {
          "application/json": {
            "schema": {

```

```

        "$ref": "#/components/schemas/bhk_Schema"
      }
    }
  },
  "401": {
    "$ref": "#/components/responses/Unauthorized"
  },
  "500": {
    "$ref": "#/components/responses/InternalServerError"
  }
},
"security": [
  {
    "BasicAuth": []
  }
],
"summary": "Get Bond HomeKit configuration.",
"tags": [
  "HomeKit"
]
},
"patch": {
  "requestBody": {
    "content": {
      "application/json": {
        "schema": {
          "$ref": "#/components/schemas/bhk_Schema"
        }
      }
    }
  },
  "responses": {
    "200": {
      "description": "Bond HomeKit enabled info updated."
    },
    "400": {
      "$ref": "#/components/responses/BadRequest"
    },
    "401": {
      "$ref": "#/components/responses/Unauthorized"
    },
    "500": {
      "$ref": "#/components/responses/InternalServerError"
    }
  },
  "security": [
    {
      "BasicAuth": []
    }
  ],
  "summary": "Enable or disable the Bond HomeKit integration.",
  "tags": [
    "HomeKit"
  ]
},
"delete": {
  "responses": {
    "204": {
      "description": "Bond HomeKit info reseted.\nThat means the Bond will delete all info, unpairing if paired."
    },
    "401": {
      "$ref": "#/components/responses/Unauthorized"
    },
    "500": {
      "$ref": "#/components/responses/InternalServerError"
    }
  },
  "security": [
    {
      "BasicAuth": []
    }
  ],
  "summary": "Reset Bond HomeKit configuration, unpairing if paired.",
  "tags": [
    "HomeKit"
  ]
}

```

```

    }
  },
  "/v2/sys/wifi/scan": {
    "get": {
      "responses": {
        "200": {
          "content": {
            "application/json": {
              "schema": {
                "$ref": "#/components/schemas/schemas_Scan"
              }
            }
          }
        },
        "description": "Scan results"
      },
      "204": {
        "description": "No results available yet"
      },
      "401": {
        "$ref": "#/components/responses/Unauthorized"
      },
      "500": {
        "$ref": "#/components/responses/InternalServerError"
      }
    },
    "security": [
      {
        "BasicAuth": []
      }
    ],
    "summary": "Scan for Wi-Fi networks",
    "description": "A GET request starts a new Wi-Fi network scan (unless a scan is already in p",
    "tags": [
      "Wi-Fi Scan"
    ]
  }
},
"/v2/sys/wifi/sta": {
  "get": {
    "responses": {
      "200": {
        "content": {
          "application/json": {
            "schema": {
              "$ref": "#/components/schemas/Sta"
            }
          }
        }
      },
      "description": "Current Wi-Fi settings"
    },
    "401": {
      "$ref": "#/components/responses/Unauthorized"
    }
  },
  "security": [
    {
      "BasicAuth": []
    }
  ],
  "summary": "Get current Wi-Fi settings",
  "description": "Get the name (base64-encoded in `ssid`) of the Wi-Fi network to which the Bor",
  "tags": [
    "Wi-Fi Station"
  ]
},
"patch": {
  "requestBody": {
    "content": {
      "application/json": {
        "schema": {
          "$ref": "#/components/schemas/Sta"
        }
      }
    }
  },
  "responses": {
    "200": {

```

```

        "content": {
          "application/json": {
            "schema": {
              "$ref": "#/components/schemas/Station"
            }
          }
        },
        "description": "New Wi-Fi settings"
      },
      "401": {
        "$ref": "#/components/responses/Unauthorized"
      },
      "500": {
        "$ref": "#/components/responses/InternalServerError"
      }
    },
    "security": [
      {
        "BasicAuth": []
      }
    ],
    "summary": "Connect to a Wi-Fi network",
    "description": "For compatibility reasons, the PATCH endpoint can also be accessed using the POST method.",
    "tags": [
      "Wi-Fi Station"
    ]
  },
  "delete": {
    "responses": {
      "204": {
        "description": "Wi-Fi network disconnected"
      },
      "401": {
        "$ref": "#/components/responses/Unauthorized"
      },
      "500": {
        "$ref": "#/components/responses/InternalServerError"
      }
    },
    "security": [
      {
        "BasicAuth": []
      }
    ],
    "summary": "Disconnect from Wi-Fi network",
    "description": "The Bond will disconnect from the current Wi-Fi network, and will not attempt to reconnect.",
    "tags": [
      "Wi-Fi Station"
    ]
  }
},
"/v2/sys/wifi/watchdog": {
  "get": {
    "responses": {
      "200": {
        "content": {
          "application/json": {
            "schema": {
              "$ref": "#/components/schemas/Watchdog"
            }
          }
        },
        "description": "Network Watchdog settings"
      },
      "401": {
        "$ref": "#/components/responses/Unauthorized"
      },
      "500": {
        "$ref": "#/components/responses/InternalServerError"
      }
    },
    "security": [
      {
        "BasicAuth": []
      }
    ],
    "summary": "Check Network Watchdog configuration",

```

```

    "tags": [
      "Network Watchdog"
    ]
  },
  "patch": {
    "requestBody": {
      "content": {
        "application/json": {
          "schema": {
            "$ref": "#/components/schemas/Watchdog"
          }
        }
      }
    },
    "responses": {
      "200": {
        "description": "Network Watchdog config updated"
      },
      "401": {
        "$ref": "#/components/responses/Unauthorized"
      },
      "500": {
        "$ref": "#/components/responses/InternalServerError"
      }
    },
    "security": [
      {
        "BasicAuth": []
      }
    ],
    "summary": "Configure Network Watchdog",
    "description": "Configure the Wi-Fi watchdog, which runs whenever the Wi-Fi station (`sta`) is connected.",
    "tags": [
      "Network Watchdog"
    ]
  }
},
"/v2/sys/wifi/ap": {
  "get": {
    "responses": {
      "200": {
        "content": {
          "application/json": {
            "schema": {
              "$ref": "#/components/schemas/Ap"
            }
          }
        }
      },
      "description": "Current Config AP settings"
    },
    "401": {
      "$ref": "#/components/responses/Unauthorized"
    }
  },
  "security": [
    {
      "BasicAuth": []
    }
  ],
  "summary": "Get Config AP settings",
  "description": "[Added in v4.27.60]\n\nGet the current Config AP (soft access point) control settings.",
  "tags": [
    "Config AP"
  ]
},
"patch": {
  "requestBody": {
    "content": {
      "application/json": {
        "schema": {
          "$ref": "#/components/schemas/Ap"
        }
      }
    }
  },
  "responses": {
    "200": {

```

```

        "content": {
          "application/json": {
            "schema": {
              "$ref": "#/components/schemas/Ap"
            }
          }
        },
        "description": "Config AP settings updated"
      },
      "401": {
        "$ref": "#/components/responses/Unauthorized"
      },
      "500": {
        "$ref": "#/components/responses/InternalServerError"
      }
    },
    "security": [
      {
        "BasicAuth": []
      }
    ],
    "summary": "Control Config AP visibility",
    "description": "[Added in v4.27.60]\n\nControl the Config AP (soft access point) behavior.\n",
    "tags": [
      "Config AP"
    ]
  },
  "/v2/sys/eth": {
    "get": {
      "responses": {
        "200": {
          "content": {
            "application/json": {
              "schema": {
                "$ref": "#/components/schemas/Eth"
              }
            }
          },
          "description": "Current Etherent settings"
        },
        "401": {
          "$ref": "#/components/responses/Unauthorized"
        },
        "404": {
          "description": "Ethernet unsupported on this product."
        }
      },
      "security": [
        {
          "BasicAuth": []
        }
      ],
      "summary": "Get current Ethernet settings",
      "description": "Note that there are seperate DNS settings for the Ethernet and Wi-Fi\ninterf",
      "tags": [
        "Ethernet"
      ]
    },
    "patch": {
      "requestBody": {
        "content": {
          "application/json": {
            "schema": {
              "$ref": "#/components/schemas/Eth"
            }
          }
        }
      },
      "responses": {
        "200": {
          "content": {
            "application/json": {
              "schema": {
                "$ref": "#/components/schemas/Eth"
              }
            }
          }
        }
      }
    }
  }
}

```

```

        },
        "description": "Ethernet settings updated"
    },
    "401": {
        "$ref": "#/components/responses/Unauthorized"
    },
    "500": {
        "$ref": "#/components/responses/InternalServerError"
    }
},
"security": [
    {
        "BasicAuth": []
    }
],
"summary": "Customize Ethernet settings",
"description": "For compatibility reasons, the PATCH endpoint can also be accessed\nusing the",
"tags": [
    "Ethernet"
]
},
"delete": {
    "responses": {
        "200": {
            "description": "Ethernet settings reset to defaults"
        },
        "401": {
            "$ref": "#/components/responses/Unauthorized"
        },
        "500": {
            "$ref": "#/components/responses/InternalServerError"
        }
    },
    "security": [
        {
            "BasicAuth": []
        }
    ],
    "summary": "Reset Ethernet Settings to Default",
    "description": "Bond will clear settings for Ethernet, defaulting to DHCP,\nDHCP-provided DNS",
    "tags": [
        "Ethernet"
    ]
}
},
"/v2/sys/upgrade": {
    "get": {
        "responses": {
            "200": {
                "content": {
                    "application/json": {
                        "schema": {
                            "$ref": "#/components/schemas/Progress"
                        }
                    }
                },
                "description": "Upgrade progress"
            },
            "204": {
                "description": "No upgrade in progress"
            },
            "401": {
                "$ref": "#/components/responses/Unauthorized"
            }
        },
        "security": [
            {
                "BasicAuth": []
            }
        ],
        "summary": "Get firmware upgrade status",
        "description": "Check the progress of a firmware upgrade.\n\nNote that when an upgrade completes",
        "tags": [
            "Upgrade"
        ]
    },
    "put": {

```

```

    "requestBody": {
      "content": {
        "application/json": {
          "schema": {
            "$ref": "#/components/schemas/Upgrade"
          }
        }
      }
    },
    "responses": {
      "200": {
        "content": {
          "application/json": {
            "schema": {
              "$ref": "#/components/schemas/Progress"
            }
          }
        },
        "description": "Progress of newly started upgrade"
      },
      "400": {
        "$ref": "#/components/responses/BadRequest"
      },
      "401": {
        "$ref": "#/components/responses/Unauthorized"
      },
      "409": {
        "description": "Upgrade already in progress"
      },
      "500": {
        "$ref": "#/components/responses/InternalServerError"
      }
    },
    "security": [
      {
        "BasicAuth": []
      }
    ],
    "summary": "Start a new firmware upgrade",
    "description": "Instruct the Bond to upgrade to the specified firmware.\n\nThe client may stop the upgrade at any time.",
    "tags": [
      "Upgrade"
    ]
  },
  "delete": {
    "responses": {
      "204": {
        "description": "No upgrade in progress (anymore)"
      },
      "401": {
        "$ref": "#/components/responses/Unauthorized"
      },
      "500": {
        "$ref": "#/components/responses/InternalServerError"
      }
    },
    "security": [
      {
        "BasicAuth": []
      }
    ],
    "summary": "Cancel any running firmware upgrade",
    "description": "The Bond will cancel any firmware upgrade in progress.\n\nCanceling a firmware upgrade will stop the upgrade immediately.",
    "tags": [
      "Upgrade"
    ]
  }
},
"/v2/sys/version": {
  "get": {
    "responses": {
      "200": {
        "content": {
          "application/json": {
            "schema": {
              "$ref": "#/components/schemas/Version"
            }
          }
        }
      }
    }
  }
}

```



```

        }
      },
      "description": "Product verison information"
    },
    "401": {
      "$ref": "#/components/responses/Unauthorized"
    },
    "500": {
      "$ref": "#/components/responses/InternalServerError"
    }
  },
  "security": [
    {
      "BasicAuth": []
    }
  ],
  "summary": "Get version info",
  "description": "Get detailed version information about hardware and firmware.\n",
  "tags": [
    "Version"
  ]
}
},
"/v2/sys/reset": {
  "put": {
    "requestBody": {
      "content": {
        "application/json": {
          "schema": {
            "$ref": "#/components/schemas/Reset"
          }
        }
      }
    }
  },
  "responses": {
    "200": {
      "description": "Reset started"
    },
    "401": {
      "$ref": "#/components/responses/Unauthorized"
    },
    "500": {
      "$ref": "#/components/responses/InternalServerError"
    }
  },
  "summary": "Reset",
  "description": "Perform a reset operation, of which there are several types:\n\n**Setup reset",
  "tags": [
    "Reset"
  ]
}
},
"/v2/sys/reboot": {
  "put": {
    "responses": {
      "200": {
        "description": "Rebooting"
      },
      "401": {
        "$ref": "#/components/responses/Unauthorized"
      },
      "500": {
        "$ref": "#/components/responses/InternalServerError"
      }
    }
  },
  "summary": "Reboot",
  "description": "Reboot Bond. No settings are modified.\n\nThis does NOT perform a factory reset",
  "tags": [
    "Reboot"
  ]
}
},
"/v2/sys/time": {
  "get": {
    "responses": {
      "200": {
        "description": "Time info",

```

```

        "content": {
          "application/json": {
            "schema": {
              "$ref": "#/components/schemas/Time"
            }
          }
        },
        "401": {
          "$ref": "#/components/responses/Unauthorized"
        },
        "404": {
          "$ref": "#/components/responses/NotFound"
        },
        "500": {
          "$ref": "#/components/responses/InternalServerError"
        }
      },
      "summary": "Get time info",
      "description": "Get information about Bond system time, timezone, and grid location.\n",
      "tags": [
        "Time"
      ]
    },
    "patch": {
      "requestBody": {
        "content": {
          "application/json": {
            "schema": {
              "$ref": "#/components/schemas/Time"
            }
          }
        }
      },
      "responses": {
        "200": {
          "description": "Time info updated"
        },
        "400": {
          "$ref": "#/components/responses/BadRequest"
        },
        "401": {
          "$ref": "#/components/responses/Unauthorized"
        },
        "404": {
          "$ref": "#/components/responses/NotFound"
        },
        "500": {
          "$ref": "#/components/responses/InternalServerError"
        }
      },
      "summary": "Modify time info",
      "description": "Set timezone, grid location, and/or time.\n\nIf `tz` is set to `null`, all s",
      "tags": [
        "Time"
      ]
    },
    "delete": {
      "responses": {
        "204": {
          "description": "Time info reset"
        },
        "401": {
          "$ref": "#/components/responses/Unauthorized"
        },
        "404": {
          "$ref": "#/components/responses/NotFound"
        },
        "500": {
          "$ref": "#/components/responses/InternalServerError"
        }
      },
      "summary": "Reset time info to defaults",
      "description": "Forget time, timezone, and grid location.\n\nAs a consequence, all skeds wil",
      "tags": [
        "Time"
      ]
    }
  ]
}

```

```

    }
  },
  "/v2/sys/power": {
    "get": {
      "responses": {
        "200": {
          "description": "Power Supply info",
          "content": {
            "application/json": {
              "schema": {
                "$ref": "#/components/schemas/Power"
              }
            }
          }
        },
        "401": {
          "$ref": "#/components/responses/Unauthorized"
        },
        "404": {
          "$ref": "#/components/responses/NotFound"
        },
        "500": {
          "$ref": "#/components/responses/InternalServerError"
        }
      },
      "summary": "Get power supply info",
      "description": "Get information about Power Supply status (which supply is active).\n\nSent (
      "tags": [
        "Power"
      ]
    }
  },
  "/v2/sys/indicate": {
    "get": {
      "responses": {
        "200": {
          "description": "Indicate status",
          "content": {
            "application/json": {
              "schema": {
                "$ref": "#/components/schemas/IndicateSchema"
              }
            }
          }
        },
        "401": {
          "$ref": "#/components/responses/Unauthorized"
        },
        "404": {
          "$ref": "#/components/responses/NotFound"
        },
        "500": {
          "$ref": "#/components/responses/InternalServerError"
        }
      },
      "summary": "Get indication status",
      "description": "[Mate Pro (MT-1500) only] [v4.26+]\n\nGet the current status of visual indic
      "tags": [
        "Indicate"
      ]
    },
    "patch": {
      "requestBody": {
        "content": {
          "application/json": {
            "schema": {
              "$ref": "#/components/schemas/IndicatePatchSchema"
            }
          }
        }
      },
      "responses": {
        "200": {
          "description": "Indicate status updated",
          "content": {
            "application/json": {
              "schema": {

```

```

        "$ref": "#/components/schemas/IndicateSchema"
      }
    }
  },
  "400": {
    "$ref": "#/components/responses/BadRequest"
  },
  "401": {
    "$ref": "#/components/responses/Unauthorized"
  },
  "404": {
    "$ref": "#/components/responses/NotFound"
  },
  "500": {
    "$ref": "#/components/responses/InternalServerError"
  }
},
"summary": "Trigger visual indication",
"description": "[Mate Pro (MT-1500) only] [v4.26+]\n\nTrigger or cancel visual indication and",
"tags": [
  "Indicate"
]
}
},
"/v2/sys/vitals": {
  "get": {
    "responses": {
      "200": {
        "description": "Device vitals",
        "content": {
          "application/json": {
            "schema": {
              "$ref": "#/components/schemas/VitalsSchema"
            }
          }
        }
      },
      "401": {
        "$ref": "#/components/responses/Unauthorized"
      },
      "404": {
        "$ref": "#/components/responses/NotFound"
      },
      "500": {
        "$ref": "#/components/responses/InternalServerError"
      }
    },
    "summary": "Get device lifetime statistics",
    "description": "[Mate Pro (MT-1500) only] [v4.28+]\n\nReturns lifetime statistics for the device",
    "tags": [
      "Vitals"
    ]
  },
  "delete": {
    "responses": {
      "204": {
        "description": "Vitals reset"
      },
      "401": {
        "$ref": "#/components/responses/Unauthorized"
      },
      "404": {
        "$ref": "#/components/responses/NotFound"
      },
      "500": {
        "$ref": "#/components/responses/InternalServerError"
      }
    },
    "summary": "Reset device lifetime statistics",
    "description": "[Mate Pro (MT-1500) only] [v4.28+]\n\nResets all vitals counters to zero and",
    "tags": [
      "Vitals"
    ]
  }
},
"/v2/sys/backup": {

```

```

    "get": {
      "responses": {
        "200": {
          "content": {
            "application/json": {
              "schema": {
                "$ref": "#/components/schemas/Backup"
              }
            }
          },
          "description": "Backup status"
        },
        "204": {
          "description": "No recent Backup/Restore."
        }
      },
      "summary": "Check Backup status",
      "description": "Shows whether a backup or a restore is running,\nincluding progress indicators",
      "tags": [
        "Backup"
      ]
    },
    "put": {
      "requestBody": {
        "content": {
          "application/json": {
            "schema": {
              "$ref": "#/components/schemas/Backup"
            }
          }
        }
      },
      "responses": {
        "200": {
          "description": "Backup/Restore started"
        },
        "401": {
          "$ref": "#/components/responses/Unauthorized"
        },
        "500": {
          "$ref": "#/components/responses/InternalServerError"
        }
      },
      "summary": "Start a Backup or Restore",
      "description": "Start a backup or restore operation,\n\nNOTE: A backup will send all devices to backup",
      "tags": [
        "Backup"
      ]
    },
    "delete": {
      "responses": {
        "204": {
          "description": "Backup/Restore canceled"
        },
        "401": {
          "$ref": "#/components/responses/Unauthorized"
        },
        "500": {
          "$ref": "#/components/responses/InternalServerError"
        }
      },
      "summary": "Cancel Backup or Restore",
      "description": "Cancel a failed operation, or clear the success/error status.\n",
      "tags": [
        "Backup"
      ]
    }
  },
  "/v2/sidekicks/{ws_id}": {
    "parameters": [
      {
        "in": "path",
        "name": "ws_id",
        "required": true,
        "schema": {
          "type": "string"
        }
      }
    ]
  }
}

```

```

    }
  ],
  "get": {
    "responses": {
      "200": {
        "content": {
          "application/json": {
            "schema": {
              "$ref": "#/components/schemas/WeatherSensor"
            }
          }
        },
        "description": "Weather Sensor object returned"
      },
      "401": {
        "$ref": "#/components/responses/Unauthorized"
      },
      "404": {
        "$ref": "#/components/responses/NotFound"
      },
      "500": {
        "$ref": "#/components/responses/InternalServerError"
      }
    },
    "summary": "Get specific Weather Sensor",
    "description": "Get information about a Weather Sensor. \n\nIn the Bond architecture, all Bond",
    "tags": [
      "Weather Sensor"
    ]
  },
  "patch": {
    "requestBody": {
      "content": {
        "application/json": {
          "schema": {
            "$ref": "#/components/schemas/WeatherSensor"
          }
        }
      }
    },
    "responses": {
      "200": {
        "description": "Sidekick modified"
      },
      "400": {
        "$ref": "#/components/responses/BadRequest"
      },
      "401": {
        "$ref": "#/components/responses/Unauthorized"
      },
      "404": {
        "$ref": "#/components/responses/NotFound"
      },
      "500": {
        "$ref": "#/components/responses/InternalServerError"
      }
    },
    "summary": "Modify an existing Weather Sensor metadata and links",
    "description": "Modifies metadata and/or links of a Weather Sensor.\nPlease, refer to Sidekick",
    "tags": [
      "Weather Sensor"
    ]
  },
  "delete": {
    "responses": {
      "204": {
        "description": "Sidekick deleted"
      },
      "401": {
        "$ref": "#/components/responses/Unauthorized"
      },
      "404": {
        "$ref": "#/components/responses/NotFound"
      },
      "500": {
        "$ref": "#/components/responses/InternalServerError"
      }
    }
  }
}

```

```

    },
    "summary": "Delete a Weather Sensor",
    "description": "Deletes specified Weather Sensor.\n",
    "tags": [
        "Weather Sensor"
    ]
}
},
"/v2/sidekicks/{ws_id}/properties": {
    "get": {
        "responses": {
            "200": {
                "content": {
                    "application/json": {
                        "schema": {
                            "$ref": "#/components/schemas/schemas_Properties"
                        }
                    }
                },
                "description": "Weather Sensor properties object"
            },
            "401": {
                "$ref": "#/components/responses/Unauthorized"
            },
            "404": {
                "$ref": "#/components/responses/NotFound"
            },
            "500": {
                "$ref": "#/components/responses/InternalServerError"
            }
        },
        "summary": "Get properties of a Weather Sensor.",
        "description": "This endpoint is used to get the current properties of a Weather Sensor.\n",
        "tags": [
            "Weather Sensor Properties"
        ]
    },
    "patch": {
        "responses": {
            "200": {
                "content": {
                    "application/json": {
                        "schema": {
                            "$ref": "#/components/schemas/schemas_Properties"
                        }
                    }
                },
                "description": "Weather Sensor properties object"
            },
            "400": {
                "$ref": "#/components/responses/BadRequest"
            },
            "401": {
                "$ref": "#/components/responses/Unauthorized"
            },
            "404": {
                "$ref": "#/components/responses/NotFound"
            },
            "500": {
                "$ref": "#/components/responses/InternalServerError"
            }
        },
        "summary": "Modify properties of a Weather Sensor.",
        "description": "This endpoint is used to modify the properties of a Weather Sensor.\nUse this",
        "tags": [
            "Weather Sensor Properties"
        ]
    },
    "parameters": [
        {
            "in": "path",
            "name": "ws_id",
            "required": true,
            "schema": {
                "type": "string"
            }
        }
    ]
}

```

```

    ],
    "/v2/sidekicks/{ws_id}/state": {
      "get": {
        "responses": {
          "200": {
            "content": {
              "application/json": {
                "schema": {
                  "$ref": "#/components/schemas/schemas_State-2"
                }
              }
            }
          },
          "description": "Get Weather Sensor state"
        },
        "401": {
          "$ref": "#/components/responses/Unauthorized"
        },
        "404": {
          "$ref": "#/components/responses/NotFound"
        },
        "500": {
          "$ref": "#/components/responses/InternalServerError"
        }
      },
      "summary": "Get Weather Sensor current state",
      "description": "Get the current state of a Weather Sensor. This object\nincludes the last mea",
      "tags": [
        "Weather Sensor State"
      ]
    },
    "parameters": [
      {
        "in": "path",
        "name": "ws_id",
        "required": true,
        "schema": {
          "type": "string"
        }
      }
    ]
  },
  "/v2/sidekicks/{ws_id}/test": {
    "put": {
      "responses": {
        "200": {
          "description": "Test/Simulation of weather event requested.",
          "content": {
            "application/json": {
              "schema": {
                "type": "object",
                "example": {
                  "event_name": "rain"
                }
              }
            }
          }
        },
        "400": {
          "$ref": "#/components/responses/BadRequest"
        },
        "401": {
          "$ref": "#/components/responses/Unauthorized"
        },
        "404": {
          "$ref": "#/components/responses/NotFound"
        },
        "500": {
          "$ref": "#/components/responses/InternalServerError"
        }
      },
      "summary": "Test/simulate weather events, triggering linked actions.",
      "description": "Execute all linked actions for the specified event type. Usefull for\nsimula",
      "tags": [
        "Weather Sensor Test"
      ]
    }
  },
},

```



```

    "parameters": [
      {
        "in": "path",
        "name": "ws_id",
        "required": true,
        "schema": {
          "type": "string"
        }
      }
    ]
  },
  "/v2/channels": {
    "get": {
      "description": "Returns a list of all channels.\n\nApplies to Mate Pro (MT-1500) and Sidekick",
      "responses": {
        "200": {
          "content": {
            "application/json": {
              "schema": {
                "$ref": "#/components/schemas/ChannelList"
              }
            }
          },
          "description": "Channel list with hashes"
        },
        "401": {
          "$ref": "#/components/responses/Unauthorized"
        },
        "404": {
          "$ref": "#/components/responses/NotFound"
        },
        "500": {
          "$ref": "#/components/responses/InternalServerError"
        }
      },
      "summary": "Get list of channels",
      "tags": [
        "Channels"
      ]
    }
  },
  "/v2/channels/{channel_id}": {
    "parameters": [
      {
        "in": "path",
        "name": "channel_id",
        "required": true,
        "schema": {
          "type": "string",
          "pattern": "^(0|[1-9]|1[0-5])$",
          "description": "Channel ID: \"0\" for All channel, or \"1\" through \"15\" for individual"
        }
      }
    ],
    "get": {
      "responses": {
        "200": {
          "content": {
            "application/json": {
              "schema": {
                "$ref": "#/components/schemas/Channel"
              }
            }
          },
          "description": "Channel object returned"
        },
        "401": {
          "$ref": "#/components/responses/Unauthorized"
        },
        "404": {
          "$ref": "#/components/responses/NotFound"
        },
        "500": {
          "$ref": "#/components/responses/InternalServerError"
        }
      },
      "summary": "Get specific channel",

```

```

        "description": "Get information about a specific channel.\n\nApplies to Mate Pro (MT-1500) and",
        "tags": [
            "Channels"
        ]
    },
    "patch": {
        "requestBody": {
            "content": {
                "application/json": {
                    "schema": {
                        "$ref": "#/components/schemas/Channel"
                    }
                }
            }
        },
        "responses": {
            "200": {
                "description": "Channel modified"
            },
            "400": {
                "$ref": "#/components/responses/BadRequest"
            },
            "401": {
                "$ref": "#/components/responses/Unauthorized"
            },
            "404": {
                "$ref": "#/components/responses/NotFound"
            },
            "500": {
                "$ref": "#/components/responses/InternalServerError"
            }
        },
        "summary": "Modify channel metadata",
        "description": "Modifies metadata of a channel, such as name, label, technology assignment,\n",
        "tags": [
            "Channels"
        ]
    }
},
"/v2/channels/{channel_id}/actions/{action_name}": {
    "put": {
        "requestBody": {
            "content": {
                "application/json": {
                    "schema": {
                        "type": "object",
                        "example": {
                            argument: 4
                        }
                    }
                }
            }
        },
        "responses": {
            "200": {
                "description": "Action executed",
                "content": {
                    "application/json": {
                        "schema": {
                            "type": "object",
                            "example": {
                                argument: 4
                            }
                        }
                    }
                }
            },
            "400": {
                "$ref": "#/components/responses/BadRequest"
            },
            "401": {
                "$ref": "#/components/responses/Unauthorized"
            },
            "404": {
                "$ref": "#/components/responses/NotFound"
            },
            "500": {

```

```

        "$ref": "#/components/responses/InternalServerError"
    },
    },
    "summary": "Execute a Channel Action",
    "description": "***[v4.24+] [MT-1500 only] Development and testing endpoint**\n\n`action_name`",
    "tags": [
        "Channel Actions"
    ],
    "security": [
        {
            "OAuth": [
                "oauth2"
            ]
        }
    ],
    "parameters": [
        {
            "in": "path",
            "name": "action_name",
            "required": true,
            "schema": {
                "type": "string"
            }
        },
        {
            "in": "path",
            "name": "channel_id",
            "required": true,
            "schema": {
                "type": "string",
                "pattern": "^(0|[1-9]|1[0-5])$"
            }
        }
    ],
    "/v2/channels/{channel_id}/skeds": {
        "parameters": [
            {
                "in": "path",
                "name": "channel_id",
                "required": true,
                "schema": {
                    "type": "string",
                    "pattern": "^(0|[1-9]|1[0-5])$",
                    "description": "Channel ID: \"0\" for All channel, or \"1\" through \"15\" for individual"
                }
            }
        ],
        "get": {
            "description": "Returns a list of sked_ids and the corresponding sked hashes for a channel.",
            "responses": {
                "200": {
                    "content": {
                        "application/json": {
                            "schema": {
                                "$ref": "#/components/schemas/SkedList"
                            }
                        }
                    },
                    "description": "Channel Schedule list returned"
                },
                "401": {
                    "$ref": "#/components/responses/Unauthorized"
                },
                "404": {
                    "$ref": "#/components/responses/NotFound"
                },
                "500": {
                    "$ref": "#/components/responses/InternalServerError"
                }
            },
            "summary": "Get list of Channel Schedules",
            "tags": [
                "Channel Schedules"
            ]
        }
    },
},

```

```

"post": {
  "requestBody": {
    "content": {
      "application/json": {
        "schema": {
          "$ref": "#/components/schemas/Sked"
        }
      }
    }
  },
  "responses": {
    "201": {
      "$ref": "#/components/responses/Created"
    },
    "400": {
      "$ref": "#/components/responses/BadRequest"
    },
    "401": {
      "$ref": "#/components/responses/Unauthorized"
    },
    "500": {
      "$ref": "#/components/responses/InternalServerError"
    }
  },
  "summary": "Create new Channel Schedule",
  "description": "Creates a Channel Schedule for a specific channel on Mate Pro (MT-1500).\n\n",
  "tags": [
    "Channel Schedules"
  ]
},
"/v2/channels/{channel_id}/skeds/{sked_id}": {
  "parameters": [
    {
      "in": "path",
      "name": "channel_id",
      "required": true,
      "schema": {
        "type": "string",
        "pattern": "^(0|[1-9]|1[0-5])$",
        "description": "Channel ID: \"0\" for All channel, or \"1\" through \"15\" for individual"
      }
    },
    {
      "in": "path",
      "name": "sked_id",
      "required": true,
      "schema": {
        "type": "string"
      }
    }
  ],
  "get": {
    "responses": {
      "200": {
        "content": {
          "application/json": {
            "schema": {
              "$ref": "#/components/schemas/Sked"
            }
          }
        },
        "description": "Channel Schedule object returned"
      },
      "401": {
        "$ref": "#/components/responses/Unauthorized"
      },
      "404": {
        "$ref": "#/components/responses/NotFound"
      },
      "500": {
        "$ref": "#/components/responses/InternalServerError"
      }
    },
    "summary": "Get specific Channel Schedule",
    "description": "Get information about a Channel Schedule on Mate Pro (MT-1500).\n",
    "tags": [

```

```

        "Channel Schedules"
    ],
    "patch": {
        "requestBody": {
            "content": {
                "application/json": {
                    "schema": {
                        "$ref": "#/components/schemas/Sked"
                    }
                }
            }
        },
        "responses": {
            "200": {
                "description": "Channel Schedule modified"
            },
            "400": {
                "$ref": "#/components/responses/BadRequest"
            },
            "401": {
                "$ref": "#/components/responses/Unauthorized"
            },
            "404": {
                "$ref": "#/components/responses/NotFound"
            },
            "500": {
                "$ref": "#/components/responses/InternalServerError"
            }
        },
        "summary": "Modify an existing Channel Schedule",
        "description": "Modifies any fields of an existing Channel Schedule on Mate Pro (MT-1500).\n",
        "tags": [
            "Channel Schedules"
        ]
    },
    "delete": {
        "responses": {
            "204": {
                "description": "Channel Schedule deleted"
            },
            "401": {
                "$ref": "#/components/responses/Unauthorized"
            },
            "404": {
                "$ref": "#/components/responses/NotFound"
            },
            "500": {
                "$ref": "#/components/responses/InternalServerError"
            }
        },
        "summary": "Delete a Channel Schedule",
        "description": "Deletes specified Channel Schedule on Mate Pro (MT-1500).\nNaturally, the Ch",
        "tags": [
            "Channel Schedules"
        ]
    }
},
"/v2/keys": {
    "get": {
        "responses": {
            "200": {
                "content": {
                    "application/json": {
                        "schema": {
                            "$ref": "#/components/schemas/KeyConfiguration"
                        }
                    }
                },
                "description": "Key configuration returned"
            },
            "401": {
                "$ref": "#/components/responses/Unauthorized"
            },
            "404": {
                "$ref": "#/components/responses/NotFound"
            }
        },
    },

```

```

        "500": {
            "$ref": "#/components/responses/InternalServerError"
        }
    },
    "summary": "Get Quick Action Button configuration",
    "description": "Get the current configuration for all three Quick Action Buttons (K1, K2, K3)",
    "tags": [
        "Quick Action Keys"
    ]
},
"patch": {
    "requestBody": {
        "content": {
            "application/json": {
                "schema": {
                    "$ref": "#/components/schemas/KeyConfiguration"
                }
            }
        }
    },
    "responses": {
        "200": {
            "description": "Key configuration modified"
        },
        "400": {
            "$ref": "#/components/responses/BadRequest"
        },
        "401": {
            "$ref": "#/components/responses/Unauthorized"
        },
        "404": {
            "$ref": "#/components/responses/NotFound"
        },
        "500": {
            "$ref": "#/components/responses/InternalServerError"
        }
    },
    "summary": "Modify Quick Action Button configuration",
    "description": "Modify the configuration for Quick Action Buttons on Mate Pro (MT-1500).\\n\\n",
    "tags": [
        "Quick Action Keys"
    ]
}
},
"x-tagGroups": [
    {
        "name": "Devices",
        "tags": [
            "Devices",
            "Device State",
            "Device Properties",
            "Device Actions",
            "Device Remote Address",
            "Device Reload",
            "Power Cycle State"
        ]
    },
    {
        "name": "Commands",
        "tags": [
            "Device Commands",
            "Device Command Signal",
            "Device Command Transmit"
        ]
    },
    {
        "name": "Groups",
        "tags": [
            "Groups",
            "Group Devices",
            "Group State",
            "Group Properties",
            "Group Actions"
        ]
    }
]

```

```

    "name": "Scenes",
    "tags": [
        "Scenes"
    ]
},
{
    "name": "Schedules",
    "tags": [
        "Device Schedules",
        "Group Schedules",
        "Scene Schedules",
        "Channel Schedules"
    ]
},
{
    "name": "Virtual Remote Control",
    "tags": [
        "Virtual Remote Control"
    ]
},
{
    "name": "Ports",
    "tags": [
        "Ports"
    ]
},
{
    "name": "Sidekick",
    "tags": [
        "Sidekick",
        "Sidekick Learn",
        "Keystream",
        "Channels",
        "Channel Actions",
        "Quick Action Keys"
    ]
},
{
    "name": "Weather Sensor",
    "tags": [
        "Weather Sensor",
        "Weather Sensor Properties",
        "Weather Sensor State",
        "Weather Sensor Test"
    ]
},
{
    "name": "Bridge",
    "tags": [
        "Bridge"
    ]
},
{
    "name": "Signals",
    "tags": [
        "Transmit",
        "Scan",
        "Scan Result",
        "Rssi"
    ]
},
{
    "name": "API",
    "tags": [
        "Token",
        "MQTT",
        "BPUP",
        "HomeKit"
    ]
},
{
    "name": "Network",
    "tags": [
        "Wi-Fi Scan",
        "Wi-Fi Station",
        "Config AP",
        "Ethernet",

```

```

        "Network Watchdog"
    ],
    {
        "name": "System",
        "tags": [
            "Upgrade",
            "Version",
            "Backup",
            "Reset",
            "Reboot",
            "Time",
            "Power",
            "Locale",
            "Indicate",
            "Vitals"
        ]
    },
    {
        "name": "Debug",
        "tags": [
            "Database",
            "LiveLog",
            "LEDs",
            "WiFi",
            "RF Manager"
        ]
    }
],
"components": {
    "schemas": {
        "Schema": {
            "type": "object",
            "properties": {
                "log_signals": {
                    "example": false,
                    "type": "boolean"
                },
                "silence_tx": {
                    "example": false,
                    "type": "boolean"
                }
            }
        }
    },
    "Leds": {
        "type": "object",
        "properties": {
            "n": {
                "example": 5,
                "type": "number",
                "description": "Number of LEDs present.\n",
                "readOnly": true
            },
            "manual": {
                "example": 1,
                "type": "number",
                "description": "- `0` : LEDs behave normally (default)\n- `1` : LED automatic control disabled\n"
            },
            "value": {
                "example": "FF000000FF000000FFFFFFFF000000",
                "type": "string",
                "description": "Concatinated 24-bit RGB hex values for each of the `n` LEDs.\n\nIn the example, the first 8 LEDs are red, the next 8 are green, and the last 8 are blue."
            }
        }
    },
    "LiveLog": {
        "type": "object",
        "properties": {
            "ip": {
                "example": "192.168.1.102",
                "type": "string",
                "description": "IPv4 address of the UDP receiver.\n"
            },
            "port": {
                "example": 1337,
                "type": "number",
                "description": "UDP port of the receiver. Must be non-zero.\n"
            }
        }
    }
}

```



```

    },
    "keyphrase": {
      "example": "correct horse battery staple",
      "type": "string",
      "description": "[v4.10.0+] Keyphrase used to derive the AES-128 encryption key for\nthe c
    }
  }
},
"wifi": {
  "type": "object",
  "properties": {
    "shutdown": {
      "example": 1,
      "type": "number"
    }
  }
},
"BeauDb": {
  "type": "object",
  "properties": {
    "live_records": {
      "example": 20,
      "type": "string",
      "description": "Number of \"live\" records in the database.\n"
    },
    "dead_records": {
      "example": 33,
      "type": "string",
      "description": "Number of \"dead\" records in the database.\nThese are records marked fo
    },
    "empty_pages": {
      "example": 0,
      "type": "string",
      "description": "Number of pages which are entirely empty.\nThis will be large initially,
    },
    "total_dead": {
      "example": 110824,
      "type": "string",
      "description": "Total amount of memory used by dead records.\nDead records are counted a
    },
    "total_free": {
      "example": 900421,
      "type": "string",
      "description": "Total free space in database (including dead space), in bytes.\n"
    },
    "max_free": {
      "example": 4002,
      "type": "string",
      "description": "Maximum freespace of any page, in bytes,\nexcepting the swap page (a sing
    },
    "total_head": {
      "example": 900421,
      "type": "string",
      "description": "Total free space in database (including dead space), in bytes.\n"
    },
    "min_head": {
      "example": 15,
      "type": "string",
      "description": "Minimum amount of headroom of any page, in bytes.\nThis dictates the sma
    },
    "compactions": {
      "example": 1,
      "type": "string",
      "description": "Number of compactions performed since boot.\n"
    },
    "size": {
      "example": 1048576,
      "type": "string",
      "description": "Total size of database, in bytes.\n"
    },
    "page_size": {
      "example": 4096,
      "type": "string",
      "description": "Size of each page, in bytes.\n"
    },
    "dirty": {
      "example": false,

```

```

        "type": "boolean",
        "description": "Indicates that a write failed and database is in an inconsistant state.\n"
    },
    },
    "Bridge": {
        "properties": {
            "location": {
                "example": "My Room",
                "type": "string"
            },
            "name": {
                "example": "My Bridge",
                "type": "string"
            },
            "bluelight": {
                "example": 127,
                "type": "integer",
                "description": "Amount of blue light emitted from the Bond when it's idle: at `0` the li
            },
            "frequencies": {
                "readOnly": true,
                "example": [
                    [
                        38,
                        38
                    ],
                    [
                        433050,
                        434079
                    ]
                ],
                "type": "array",
                "items": {
                    "type": "array",
                    "items": {
                        "type": "integer"
                    }
                },
                "description": "List of intervals (inclusive) of supported frequencies,\nin kHz. Values
            }
        }
    },
    "ErrorBaseSchema": {
        "properties": {
            "_error_id": {
                "example": 1,
                "type": "number"
            }
        }
    },
    "Unauthorized": {
        "allOf": [
            {
                "$ref": "#/components/schemas/ErrorBaseSchema"
            },
            {
                "properties": {
                    "_error_msg": {
                        "example": "You need authentication credentials to continue",
                        "type": "string"
                    }
                },
                "type": "object"
            }
        ]
    },
    "NotFound": {
        "allOf": [
            {
                "$ref": "#/components/schemas/ErrorBaseSchema"
            },
            {
                "properties": {
                    "_error_msg": {
                        "example": "Resource not found",
                        "type": "string"
                    }
                }
            }
        ]
    }
}

```

```

    },
    "type": "object"
  }
],
},
"InternalServerError": {
  "allof": [
    {
      "$ref": "#/components/schemas/ErrorBaseSchema"
    },
    {
      "properties": {
        "_error_msg": {
          "example": "Something unexpected happened (this field will have a description of the error)",
          "type": "string"
        }
      },
      "type": "object"
    }
  ],
  "type": "object"
},
],
},
"Locale": {
  "properties": {
    "compliance_region": {
      "example": "au",
      "type": "string",
      "description": "Region for radio frequency compliance.\nThis option must be set by the country of origin."
    },
    "compliance_regions": {
      "example": [
        "us",
        "au",
        "il",
        "eu"
      ],
      "type": "array",
      "items": {
        "type": "string"
      },
      "description": "[Added in v4.12]\nList of compliance regions supported by this Bond.\nSee the Compliance Regions section for more details."
    }
  },
  "type": "object"
},
},
"DeviceList": {
  "type": "object",
  "example": {
    "7fc1e84b": {
      "3b20f300": {
        "_": "9a5e1136"
      },
      "4caf6472": {
        "_": "409d124b"
      }
    }
  },
  "type": "object"
},
},
"Device": {
  "properties": {
    "name": {
      "example": "My Fan",
      "type": "string"
    },
    "type": {
      "description": "- `CF` : Ceiling Fan\n- `FP` : Fireplace\n- `HT` : Heater\n- `MS` : Motor\n- `SW` : Switch\n- `TV` : TV\n- `W` : Wireless\n- `WV` : Wireless\n- `WV` : Wireless\n- `WV` : Wireless",
      "example": "CF",
      "type": "string"
    },
    "subtype": {
      "description": "[Added in v2.21]\n\nThe subtype field is used to distinguish between categories of devices.",
      "example": "AWNING",
      "type": "string"
    },
    "template": {
      "description": "An internal name describing what sort of protocol the remote represented.",
      "example": "A1",
      "type": "string"
    }
  },
  "type": "object"
},
},

```

```

        "location": {
            "example": "Kitchen",
            "type": "string"
        },
        "actions": {
            "readOnly": true,
            "type": "array",
            "example": [
                "TurnOn",
                "TurnOff",
                "TogglePower",
                "SetSpeed",
                "IncreaseSpeed",
                "DecreaseSpeed"
            ]
        },
        "properties": {
            "readOnly": true,
            "type": "object",
            "example": {
                "_": "84cd8a43"
            }
        },
        "state": {
            "readOnly": true,
            "type": "object",
            "example": {
                "_": "ad9bcde4"
            }
        },
        "commands": {
            "readOnly": true,
            "type": "object",
            "example": {
                "_": "ad9bcde4"
            }
        }
    },
    "required": [
        "name",
        "type"
    ]
},
"Created": {
    "properties": {
        "_id": {
            "example": "01234567",
            "type": "string",
            "description": "id of new device"
        }
    },
    "type": "object"
},
"BadRequest": {
    "allOf": [
        {
            "$ref": "#/components/schemas/ErrorBaseSchema"
        },
        {
            "properties": {
                "_error_msg": {
                    "example": "The resource wasn't created because of a validation problem (This field v",
                    "type": "string"
                }
            }
        }
    ],
    "type": "object"
}
],
"Conflict": {
    "allOf": [
        {
            "$ref": "#/components/schemas/ErrorBaseSchema"
        },
        {
            "properties": {
                "_error_msg": {

```

```

        "example": "The resource wasn't created because of a field conflict",
        "type": "string"
    },
    "type": "object"
}
]
},
"State": {
    "example": {
        "breeze": [
            1,
            0.2,
            0.9
        ],
        "brightness": 75,
        "light": 1,
        "power": 0,
        "speed": 2,
        "timer": 3599
    },
    "properties": {
        "_lock_priority": {
            "type": "number",
            "description": "Device lock priority number.\n"
        },
        "_lock_expiration": {
            "type": "number",
            "description": "Remaining time for expiration of the lock, in milliseconds.\n"
        }
    }
},
"RemoteAddr": {
    "properties": {
        "addr": {
            "example": [
                "dead",
                "0f2a",
                "0002",
                "0003"
            ],
            "type": "array",
            "items": {
                "type": "string"
            },
            "description": "Learned remote address(es) in hexadecimal,\nmay be patched only with a s
        },
        "addr_slots": {
            "example": 5,
            "type": "integer",
            "readOnly": true,
            "description": "Maximum number of addresses which can be learned by this device.\nTypical
        },
        "learn_window_ms": {
            "example": 60000,
            "type": "integer",
            "readOnly": true,
            "description": "Duration of learn window (milliseconds).\n"
        },
        "learn_window_remaining_ms": {
            "example": 1100,
            "type": "integer",
            "readOnly": true,
            "description": "Milliseconds remaining before learn window closes.\n"
        },
        "learn_window_open": {
            "example": true,
            "type": "boolean",
            "description": "Whether new address can be learned via RF.\nTrue iff learn_window_remain
        },
        "learn_count": {
            "example": 0,
            "type": "integer",
            "readOnly": true,
            "description": "Number of times a new address has been learned on this boot.\nUseful in t
        },
        "addr0_is_dip": {

```

```

        "example": true,
        "type": "boolean",
        "description": "If present and true, indicates that the receiver has DIP switches\nwhich
    }
}
},
"Properties": {
    "properties": {
        "trust_state": {
            "example": false,
            "type": "boolean",
            "description": "Whether or not the Bond should trust its toggleable state belief.\nThis
        },
        "addr": {
            "readOnly": true,
            "example": "10101",
            "type": "string",
            "description": "(Bridge-only)"
        },
        "freq": {
            "readOnly": true,
            "example": 434300,
            "type": "integer",
            "description": "(Bridge-only)"
        },
        "bps": {
            "readOnly": true,
            "example": 3000,
            "type": "integer",
            "description": "(Bridge-only)"
        },
        "zero_gap": {
            "readOnly": true,
            "example": 30,
            "type": "integer",
            "description": "(Bridge-only)"
        }
    }
},
"PowerCycleState": {
    "properties": {
        "enabled": {
            "example": false,
            "type": "boolean",
            "description": "If true, device will revert to specified state after power loss.\nIf fal
        },
        "state": {
            "example": {
                "light": 1,
                "brightness": 100,
                "power": 0,
                "speed": 3
            },
            "type": "object",
            "description": "State to revert to after power loss.\nSame schema as for Device state.\n
        }
    }
},
"Args": {
    "type": "object",
    "properties": {
        "field": {
            "type": "string",
            "example": "speed"
        },
        "value": {
            "type": "string",
            "example": "70%"
        }
    }
},
"Locked": {
    "allof": [
        {
            "$ref": "#/components/schemas/ErrorBaseSchema"
        },
        {

```

```

        "properties": {
            "_error_msg": {
                "example": "Device is locked with non expired higher priority",
                "type": "string"
            }
        },
        "type": "object"
    }
]
},
"CommandList": {
    "type": "object",
    "example": {
        "_": "7fc1e84b",
        "3b20f300": {
            "_": "9a5e1136"
        },
        "4ad874d2": {
            "_": "84819a9f"
        },
        "a94328bc": {
            "_": "23141efa"
        },
        "d87d6ef2": {
            "_": "2425a8bc"
        }
    }
},
"Command": {
    "type": "object",
    "properties": {
        "name": {
            "example": "Light",
            "type": "string",
            "description": "Name of corresponding button displayed in Bond App\n"
        },
        "action": {
            "example": "SetSpeed",
            "type": "string",
            "description": "Action which should be triggered when button is pressed.\n"
        },
        "argument": {
            "example": 3,
            "nullable": true,
            "description": "Argument of `action`. If absent, no argument will be used.\nMay be any JS value"
        },
        "icon": {
            "example": "speed_3",
            "type": "string",
            "description": "Icon slug. (This is the base of the name of the icon to use to represent the command)"
        },
        "category_name": {
            "example": "fan",
            "type": "string",
            "description": "The category in which to group this command.\n"
        },
        "button_type": {
            "example": "tap",
            "type": "string",
            "description": "The type of button to use for this command.\n"
        },
        "hidden": {
            "example": true,
            "type": "boolean",
            "description": "Whether to hide this command in the mobile app. Useful for exposing an\n"
        }
    },
    "required": [
        "name",
        "action"
    ]
},
"Signal": {
    "required": [
        "freq",
        "data",
        "bps",

```

```

    "encoding"
  ],
  "type": "object",
  "properties": {
    "freq": {
      "example": 434000,
      "type": "number",
      "description": "Frequency in kHz:\n - `>= 1000` is RF\n - `< 1000` is IR\n\nBond Bridge",
    },
    "modulation": {
      "example": "00K",
      "type": "string",
      "description": "- `00K` = on-off keying\n- `GFSK` = gaussian frequency-shift keying\n\nIR",
    },
    "data": {
      "example": "110100110110H",
      "type": "string",
      "description": "String representation of data bits to be transmitted.\n\nThe maximum length",
    },
    "encoding": {
      "example": "cq",
      "type": "string",
      "description": "Specifies the encoding of the data string.\n\nIn all cases, the duration of the",
    },
    "bps": {
      "example": 1000,
      "type": "number",
      "description": "Bitrate of `data`. Range is 100 to 40000.\n",
    },
    "reps": {
      "example": 12,
      "type": "number",
      "description": "Number of times that data should be repeated.\n\nIf not provided, `1` is used",
    },
    "use_scan": {
      "example": true,
      "type": "boolean",
      "writeOnly": true,
      "description": "If present and true, the Signal recorded by the most recent scan will\n\nbe",
    }
  }
},
"SkedList": {
  "type": "object",
  "example": {
    "7fc1e84b": {
      "3b20f300": {
        "_": "9a5e1136"
      },
      "4ad874d2": {
        "_": "84819a9f"
      },
      "a94328bc": {
        "_": "23141efa"
      },
      "d87d6ef2": {
        "_": "2425a8bc"
      }
    }
  }
},
"Sked": {
  "type": "object",
  "properties": {
    "enabled": {
      "example": true,
      "type": "boolean",
      "description": "If false, schedule will never execute, but will remain in Bond database.",
    },
    "action": {
      "example": "SetBrightness",
      "type": "string",
      "description": "Action which will be run on actor when schedule executes.\n\nRequired for IR",
    },
    "argument": {
      "example": 80,
      "description": "Argument to action, if needed.\n"
    }
  }
},

```



```

    "seconds": {
      "example": -3600,
      "type": "integer",
      "description": "Time at which schedule should execute, relative to `mark`,\nin seconds. ",
    },
    "days_of_week": {
      "example": [
        false,
        true,
        true,
        true,
        true,
        true,
        false
      ],
      "type": "array",
      "items": {
        "type": "boolean"
      },
      "description": "Days of week on which the schedule should execute.\n\nFirst element is f",
    },
    "mark": {
      "example": "sunset",
      "type": "string",
      "description": "Event which serves as the benchmark for the daily relative time.\nOptions",
    },
  },
  "required": [
    "action",
    "seconds",
    "mark",
    "days_of_week"
  ],
},
"vrc": {
  "properties": {
    "enabled": {
      "example": true,
      "type": "boolean",
      "description": "When enabled, the `vrc/keystream` endpoint becomes available.\n\nWhen ena",
    }
  }
},
"vrcKeystream": {
  "type": "object",
  "properties": {
    "seq": {
      "example": 3210,
      "type": "integer",
      "description": "Sequence number which client should increments for request.\n\nDESIGN NO",
    },
    "event": {
      "example": "HOLD",
      "type": "string",
      "description": "Event which occured on the specified key. Possibilities are:\n - TAP --",
    },
    "key": {
      "example": [
        1,
        3
      ],
      "type": "array",
      "description": "list of 1-based key numbers. Must be non-empty.\n\nIn the case of a sing",
    },
    "hold_ms": {
      "example": 1760,
      "type": "integer",
      "description": "Length of time for which key(s) has been held, in milliseconds.\nOnly pro",
    }
  }
},
"GroupList": {
  "type": "object",
  "example": {
    "_": "7fc1e84b",
    "3b20f300": {
      "_": "9a5e1136"
    }
  }
}

```

```

    },
    "4caf6472": {
      "_": "409d124b"
    }
  },
  "Group": {
    "properties": {
      "name": {
        "example": "Kitchen Shades",
        "type": "string"
      },
      "devices": {
        "description": "List of member Device IDs.\n\nThis field may be set during Group creation",
        "type": "array",
        "example": [
          "aabbccdd",
          "11223344",
          "deadbeef"
        ]
      },
      "types": {
        "readOnly": true,
        "description": "For values, see `type` field of Device schema.\n\nType is automatically set",
        "example": [
          "MS"
        ],
        "type": "array"
      },
      "locations": {
        "readOnly": true,
        "example": [
          "Kitchen"
        ],
        "type": "array",
        "description": "Similar to `types`, if all Devices in the Group are in the same Location"
      },
      "actions": {
        "readOnly": true,
        "type": "array",
        "example": [
          "Open",
          "Close",
          "Raise",
          "Lower",
          "Retract",
          "Extend",
          "Preset"
        ],
        "description": "The list of available Actions on the Group.\n\nThis is the intersection of"
      },
      "state": {
        "readOnly": true,
        "type": "object",
        "example": {
          "_": "ad9bcde4"
        }
      }
    },
    "required": [
      "name"
    ]
  },
  "schemas_State": {
    "example": {
      "open": 1
    }
  },
  "SceneList": {
    "type": "object",
    "example": {
      "_": "7fc1e84b",
      "3b20f300": {
        "_": "9a5e1136"
      }
    },
    "4caf6472": {
      "_": "409d124b"
    }
  }
}

```

```

    }
  },
  "Scene": {
    "type": "object",
    "properties": {
      "name": {
        "example": "Privacy",
        "type": "string"
      },
      "actors": {
        "description": "Array of actions to be executed when the Scene is run.\n\nThis field must be an array of objects, each containing a device ID, an action name, and optional arguments.\n\nExample: [\n  {\n    \"device\": \"aabbccdd\",\n    \"action\": \"TurnOn\"\n  },\n  {\n    \"group\": \"11223344\",\n    \"action\": \"SetBrightness\",\n    \"argument\": 50\n  }\n]\n",
        "type": "array",
        "example": [
          {
            "device": "aabbccdd",
            "action": "TurnOn"
          },
          {
            "group": "11223344",
            "action": "SetBrightness",
            "argument": 50
          }
        ]
      },
      "types": {
        "readOnly": true,
        "description": "For values, see `type` field of Device schema.\n\nAutomatically set by the Scene.\n",
        "example": [
          "MS"
        ],
        "type": "array"
      },
      "locations": {
        "readOnly": true,
        "example": [
          "Kitchen"
        ],
        "type": "array",
        "description": "Similar to `types`, if all Devices in the Scene are in the same Location.\n"
      },
      "imported_from": {
        "description": "If the Scene was imported from a Managed Technology (that is, a partner company).\n",
        "type": "string",
        "example": "powerview"
      }
    },
    "required": [
      "name"
    ]
  },
  "PortList": {
    "type": "object",
    "description": "Object keyed by port ID, with each entry containing the port hash\n\nin the `ports` field of the Device schema.\n",
    "example": {
      "1": {
        "_": "a1b2c3d4"
      },
      "2": {
        "_": "e5f6a7b8"
      },
      "_": "f0e1d2c3"
    }
  },
  "PortsMeta": {
    "type": "object",
    "description": "Enum values supported by this firmware build for the fields of a\n\nport. Each field has a list of supported values.\n",
    "properties": {
      "port_type": {
        "type": "array",
        "items": {
          "type": "string"
        }
      },
      "example": [
        "LT",
        "MS"
      ]
    },
    "required": [
      "port_type"
    ]
  }
}

```

```

    "description": "All `port_type` discriminator values supported by this firmware\nbuild.",
  },
  "port_class": {
    "type": "array",
    "items": {
      "type": "string"
    },
    "example": [
      "PIXEL",
      "AURA",
      "LINEAR"
    ],
    "description": "All `port_class` discriminator values supported by this firmware\nbuild.",
  },
  "link_mode": {
    "type": "array",
    "items": {
      "type": "string"
    },
    "example": [
      "synchro"
    ],
    "description": "Values accepted by `PATCH /v2/ports/{port_id}` for `link_mode`. \nDay-1 f.",
  },
  "aura_mode": {
    "type": "array",
    "items": {
      "type": "string"
    },
    "example": [
      "rgbw",
      "rgb",
      "rgbwc"
    ],
    "description": "Values accepted by `PATCH` for an AURA port's `aura_mode` field. \nReflec",
  },
  "pixel_proto": {
    "type": "array",
    "items": {
      "type": "string"
    },
    "example": [
      "ws2811",
      "ws2812b",
      "ws2814",
      "sk6812",
      "tm1903"
    ],
    "description": "Values accepted by `PATCH` for a PIXEL port's `pixel_proto` \nfield. Refl",
  },
  "pixel_order": {
    "type": "object",
    "example": {
      "ws2811": "rgb",
      "ws2812b": "rgbw",
      "ws2814": "rgbw",
      "sk6812": "rgbw",
      "tm1903": "rgb"
    },
    "description": "Map keyed by `pixel_proto`. Each value is a string of allowed \nchannel c",
  },
  "example": {
    "port_type": [
      "LT",
      "MS"
    ],
    "port_class": [
      "PIXEL",
      "AURA",
      "LINEAR"
    ],
    "link_mode": [
      "synchro"
    ],
    "aura_mode": [
      "rgbw",

```

```

        "rgb",
        "rgbc"
    ],
    "pixel_proto": [
        "ws2811",
        "ws2812b",
        "ws2814",
        "sk6812",
        "tm1903"
    ],
    "pixel_order": {
        "ws2811": "rgb",
        "ws2812b": "rgbc",
        "ws2814": "rgbc",
        "sk6812": "rgbc",
        "tm1903": "rgb"
    }
},
"Port": {
    "type": "object",
    "description": "A physical electrical port (LED strip, PWM channel group, motor\nH-bridge) with\nproperties",
    "properties": {
        "port_type": {
            "readOnly": true,
            "type": "string",
            "example": "LT",
            "description": "Port type. One of:\n - `LT` : Light\n - `MS` : Motor\n"
        },
        "port_class": {
            "readOnly": true,
            "type": "string",
            "example": "PIXEL",
            "description": "Port class. One of:\n - `PIXEL` : Addressable LED strip\n - `AURA` : Aura\n"
        },
        "enabled": {
            "type": "boolean",
            "example": true,
            "description": "Whether the port is active. Disabling a port removes its\nunderlying device\n"
        },
        "link_mode": {
            "type": "string",
            "enum": [
                "indep",
                "synchro"
            ],
            "example": "indep",
            "description": "Link mode for ports that support linking. When set to `synchro`\non one port, all\nports in the group must be set to the same mode\n"
        },
        "devices": {
            "readOnly": true,
            "type": "array",
            "items": {
                "type": "string"
            },
            "example": [
                "0000000000000001"
            ],
            "description": "64-bit hex device IDs (at `/v2/devices/{id}`) that this port\ncurrently controls\n"
        },
        "pixel_proto": {
            "type": "string",
            "enum": [
                "ws2811",
                "ws2812b",
                "ws2814",
                "sk6812",
                "tm1903"
            ],
            "example": "ws2811",
            "description": "PIXEL ports only – addressable LED protocol. Only present on\nports of class\nPIXEL\n"
        },
        "pixel_order": {
            "type": "string",
            "enum": [
                "rgb",
                "grb",
                "rbg",
                "brg",
                "gbr",
                "brg"
            ]
        }
    }
}

```

```

        "bgr",
        "rbg",
        "gbr",
        "brg"
    ],
    "example": "grb",
    "description": "PIXEL ports only – color channel order. Only present on ports\nof class",
    },
    "aura_mode": {
        "type": "string",
        "example": "rgbwc",
        "description": "AURA ports only – target-dependent PWM channel mode (e.g.\n`rgb`, `rgbw`",
    }
    },
    "example": {
        "port_type": "LT",
        "port_class": "PIXEL",
        "enabled": true,
        "link_mode": "indep",
        "pixel_proto": "ws2811",
        "pixel_order": "grb",
        "devices": [
            "0000000000000001"
        ]
    }
    },
    "SidekickList": {
        "type": "object",
        "example": {
            "_": "7fc1e84b",
            "3b20f300": {
                "_": "9a5e1136"
            },
            "4ad874d2": {
                "_": "84819a9f"
            },
            "a94328bc": {
                "_": "23141efa"
            },
            "d87d6ef2": {
                "_": "2425a8bc"
            }
        }
    },
    "Sidekick": {
        "type": "object",
        "properties": {
            "_id": {
                "writeOnly": true,
                "example": "SNAXX12345",
                "type": "string",
                "description": "Provided only on POST request to specify the Sidekick ID.\n"
            },
            "name": {
                "example": "My Sidekick",
                "type": "string",
                "description": "Name of Sidekick on Bond Home app and suggested name for \ncontrol system",
            },
            "location": {
                "example": "My Room",
                "type": "string",
                "description": "Location of Sidekick.\n"
            },
            "keys": {
                "readOnly": true,
                "example": 3,
                "type": "integer",
                "description": "Number of keys available on this Sidekick.\nMay change after creation due",
            },
            "chans": {
                "readOnly": true,
                "example": 5,
                "type": "integer",
                "description": "Number of channels available on this Sidekick.\nMay change after creation",
            },
            "chan_links": {
                "example": {

```

```

    "1": [
      {
        "device": "aabbccdd",
        "direct": 1
      },
      {
        "device": "11223344",
        "direct": 0
      }
    ],
    "2": [],
    "3": [],
    "4": [],
    "5": []
  },
  "type": "object",
  "description": "Object describes every link between channels of this Sidekick and the\n\n",
  "key_links": {
    "example": {
      "1": [
        {
          "event": "TAP",
          "channel": 5,
          "action": "Raise"
        }
      ],
      "2": [
        {
          "event": "DOUBLE_TAP",
          "device": "aabbccdd",
          "action": "SetPosition",
          "argument": 75
        }
      ],
      "3": [
        {
          "event": "TAP",
          "channel": -1,
          "action": "Stop"
        }
      ]
    }
  },
  "type": "object",
  "description": "Object describes every link between key events from this Sidekick and the\ndevice, group, scene, or channel actions which they trigger.\n\n",
  "For Mate Pro (MT-1500), key_links are used for \"built-in scenes\" - programmable\nactions that can target channels on the Mate itself, in addition to BondSync\nintegration with Bond Bridge devices, groups, and scenes.\n\n",
  "Each key number maps to an array, each element of which is called\n\"key link\". Each\n",
  "signal": {
    "readOnly": true,
    "example": 97,
    "type": "integer",
    "description": "Signal quality indication, range 1-100.\nNote that, to avoid hash thrash\n",
  },
  "battery": {
    "readOnly": true,
    "example": 80,
    "type": "integer",
    "description": "Sidekick battery remaining, range 1-100.\nValues may be limited to:\n -\n",
  },
  "model": {
    "readOnly": true,
    "example": "SKN-386",
    "type": "string",
    "description": "User-facing model number of Sidekick.\n"
  }
},
  "keystream": {
    "type": "object",
    "properties": {
      "seq": {
        "example": 3210,

```

```

        "type": "integer",
        "description": "Sequence number which increments each time this Sidekick transmits\nna Bo
    },
    "event": {
        "example": "TAP",
        "type": "string",
        "description": "Event which occurred on the specified key. Possibilities are:\n - TAP\n
    },
    "key": {
        "example": 3,
        "type": "integer",
        "description": "1-based key number.\n"
    },
    "hold_ms": {
        "example": 1760,
        "type": "integer",
        "description": "Length of time for which key has been held, in milliseconds.\nOnly provi
    }
}
},
"Learn": {
    "type": "object",
    "properties": {
        "learn_window_ms": {
            "example": 60000,
            "type": "integer",
            "readOnly": true,
            "description": "Duration of learn window (milliseconds).\n"
        },
        "learn_window_remaining_ms": {
            "example": 1100,
            "type": "integer",
            "readOnly": true,
            "description": "Milliseconds remaining before learn window closes.\n"
        },
        "learn_window_open": {
            "example": true,
            "type": "boolean",
            "description": "Whether new Sidekick can be learned.\nTrue iff learn_window_remaining_ms
        },
        "learn_count": {
            "example": 0,
            "type": "integer",
            "readOnly": true,
            "description": "Number of times a new Sidekick has been learned on this boot.\nUseful in
        },
        "new_id": {
            "example": "SHXXX12345",
            "type": "string",
            "readOnly": true,
            "description": "ID of the newly learned Sidekick, if any.\nPopulated when a new Sidekick
        },
        "pattern": {
            "example": "^SH",
            "type": "string",
            "writeOnly": true,
            "maxLength": 32,
            "description": "[v4.24+] Optional regex pattern to filter devices during learning (max 3
        }
    }
},
"Rssi": {
    "example": {
        "format": [
            "freq",
            "rssi"
        ],
        "results": [
            [
                350000,
                80
            ],
            [
                351000,
                87
            ],
            [

```



```

        352000,
        92
    ]
}
},
"type": "object"
},
"Scan": {
    "type": "object",
    "properties": {
        "freq": {
            "example": 434000,
            "type": "number",
            "description": "Frequency to scan in kHz.\n\nIf not provided, scan will search all RF fr
        },
        "modulation": {
            "example": "OOK",
            "type": "string",
            "description": "- `OOK` = on-off keying\n- `GFSK` = gaussian frequency-shift keying\n\nI
        },
        "running": {
            "example": false,
            "readOnly": true,
            "description": "If true, scan is running.\n"
        },
        "total_timeout": {
            "example": 30,
            "type": "number",
            "readOnly": true,
            "description": "The time from scan start to scan timeout, in\nseconds.\n"
        },
        "success": {
            "example": true,
            "readOnly": true,
            "description": "If true, a signal was found.\n"
        }
    }
},
"TokenGet": {
    "properties": {
        "locked": {
            "example": 0,
            "type": "integer",
            "description": "If 1, means that client must provide pin to unlock.\n\nAlternatively, the
        },
        "token": {
            "example": "8f514567acaf9869",
            "type": "string",
            "description": "Token for controlling this Bond over local network.\n\nRequired to be pr
        },
        "account_code": {
            "example": "1234567890",
            "type": "string",
            "description": "Account code previously set by PATCH.\n\nOnly provided if locked is 0.\n"
        },
        "nonce": {
            "example": "876a2b14c55b3e5a",
            "type": "string",
            "description": "Random 64-bit hex string generated whenever client sets account_code,\nne
        },
        "pin_attempts_left": {
            "example": 9,
            "type": "integer",
            "description": "Number of times which the PIN may be guessed.\n\nOnce `pin_attempts_left
        },
        "disable": {
            "example": false,
            "type": "boolean",
            "description": "If disabled, HTTP and other local APIs will not require token to function
        }
    }
},
"TokenPatch": {
    "properties": {
        "locked": {
            "example": 0,
            "type": "integer",

```

```

        "description": "If 1, means that the client wants to immediately re-lock this endpoint.\n",
    },
    "account_code": {
        "example": "1234567890",
        "type": "string",
        "description": "10-digit account code, used to specify which account to associate the Bond\n",
    },
    "pin": {
        "example": "1234",
        "type": "string",
        "description": "Pin number, if required, as part of unlocking procedure.\n"
    },
    "disable": {
        "example": false,
        "type": "boolean",
        "description": "If disabled, HTTP and other local APIs will not require token to function\n"
    }
},
},
"mqtt_schema": {
    "type": "object",
    "properties": {
        "host": {
            "example": "my-mqtt-server.myhome.com",
            "type": "string",
            "description": "IP address or domain name of the MQTT broker.\nIf set to empty string, the\n",
        },
        "port": {
            "example": 8883,
            "type": "number",
            "description": "Specify a different port number for the MQTT broker.\nIf set to 0, the Bond\n",
        },
        "cert_set": {
            "example": true,
            "type": "boolean",
            "readOnly": true,
            "description": "If true, a custom Bond certificate has been loaded.\nIf false, the Bond's\n",
        },
        "cert": {
            "example": "certificate_goes_here",
            "type": "string",
            "writeOnly": true,
            "description": "Specify a custom Bond certificate.\nTypically, this is the Bond-specific\n",
        },
        "key_set": {
            "example": true,
            "type": "boolean",
            "readOnly": true,
            "description": "If true, a custom Bond private key has been loaded.\nIf false, the Bond's\n",
        },
        "key": {
            "example": "key_goes_here",
            "type": "string",
            "writeOnly": true,
            "description": "Specify a custom Bond private key.\nWrite to empty string to go back to\n",
        },
        "server_cert_set": {
            "example": true,
            "type": "boolean",
            "readOnly": true,
            "description": "If true, a custom server CA has been loaded.\nIf false, the Bond Cloud CA\n",
        },
        "enabled": {
            "example": true,
            "type": "boolean",
            "description": "[Added in v3.3.3]\nIf true (default), Bond will attempt MQTT connection.\n",
        },
        "server_cert": {
            "example": "certificate_goes_here",
            "type": "string",
            "writeOnly": true,
            "description": "Specify a custom MQTT server CA.\nSet to empty string to return to default\n",
        },
        "server_cert_check": {
            "example": true,
            "type": "boolean",
            "description": "Set to false to disable verification of the MQTT server CA.\nIt is strong\n"
        }
    }
}

```

```

    }
  },
  "bupup_Schema": {
    "type": "object",
    "properties": {
      "broadcast": {
        "example": true,
        "type": "boolean",
        "description": "If true, the Bond will broadcast all state updates on port 30007\n(disabled)"
      }
    }
  },
  "bhk_Schema": {
    "type": "object",
    "properties": {
      "enabled": {
        "example": false,
        "type": "boolean",
        "description": "- `false`: HomeKit logic and operations will not be executed. All settings will be ignored."
      },
      "paired": {
        "example": false,
        "type": "boolean",
        "readOnly": true,
        "description": "When enabled is false, paired will always be false.\nWhen enabled is true, paired will be true if the device is paired."
      },
      "setup_code": {
        "example": "12341234",
        "type": "string",
        "readOnly": true,
        "description": "The HomeKit Setup Code is your Bond PIN two times.\nFor example, if your PIN is 1234, the setup code will be 12341234."
      }
    }
  },
  "schemas_Scan": {
    "example": {
      "hidden_requires_bssid": true,
      "format": [
        "ssid",
        "bssid",
        "auth",
        "ch",
        "signal"
      ],
      "results": [
        [
          "REIyQzk1",
          "00A423DB2C9D",
          4,
          1,
          90
        ],
        [
          "b2xpYnJhIGFpcnBvcnQgMQ==",
          "4C3275C649C4",
          3,
          11,
          74
        ],
        [
          "UGxheVNPZ2h0XzVH",
          "003E5D921274",
          4,
          6,
          73
        ],
        [
          "b2xpYnJhIG1pZmkgMw==",
          "101331C514C7",
          3,
          6,
          61
        ],
        [
          "S0VFQk9YIFcxNTB0Ug==",
          "D8EB97B1B322",
          3,
          6,
          61
        ]
      ]
    }
  }
}

```

```

0,
11,
60
],
[
  "VmVyaXpvbi1NQkhBMTAtMTRDNy1ndwVzdA==",
  "121331C514C8",
  3,
  6,
  60
],
[
  "b2xpYnJhIGFpcnBvcnQgMQ==",
  "9801A7E7ABD4",
  3,
  6,
  57
],
[
  "b3B0aW11bXdPZmlfUGFzc3BvaW50",
  "E00EDA137575",
  5,
  1,
  39
],
[
  "NkMxNDYw",
  "0072826C1468",
  4,
  1,
  36
],
[
  "UkRMRiBXaS1GaSB0ZXR3b3Jr",
  "881FA13633AA",
  3,
  1,
  33
],
[
  "SFAtUHJpbmQtRjEtT2ZmaWNlamV0IFBybyA4NjAw",
  "28924ABF85F1",
  3,
  6,
  30
],
[
  "Wm9oYXlYIyYwlycG9ydA==",
  "00F76FCDE89C",
  3,
  11,
  28
],
[
  "aWM=",
  "149182C06087",
  3,
  1,
  26
],
[
  "bWVkaWFsaW5r",
  "14358B0B1920",
  2,
  1,
  24
]
]
}
},
"Sta": {
  "properties": {
    "status": {
      "example": 2,
      "type": "number",
      "description": "Indication of whether the Wi-Fi settings have been successful:\n  - -4 =
    },
  },
},

```

```

    "ssid": {
      "example": "bGlua3N5cw==",
      "type": "string",
      "nullable": true,
      "description": "Can be `null` if it's not set."
    },
    "bssid": {
      "example": "001122334455",
      "type": "string",
      "nullable": true,
      "description": "Can be `null` if it's not set."
    },
    "bssid_set": {
      "readOnly": true,
      "example": true,
      "type": "boolean"
    },
    "mac": {
      "example": "30aea48fa8e8",
      "type": "string",
      "readOnly": true
    },
    "static_ip_set": {
      "example": false,
      "type": "boolean",
      "description": "[changed in v2.16.2]\nIf true on write, the ip, gw, netmask fields are r
    },
    "ip": {
      "example": "192.168.1.42",
      "type": "string",
      "nullable": true,
      "description": "Can be `null` if it's not connected."
    },
    "gw": {
      "example": "192.168.1.1",
      "type": "string",
      "nullable": true,
      "description": "Can be `null` if it's not connected."
    },
    "netmask": {
      "example": "255.255.255.0",
      "type": "string",
      "nullable": true,
      "description": "Can be `null` if it's not connected."
    },
    "dns_set": {
      "example": true,
      "type": "boolean",
      "description": "[changed in v2.16.2]\nIf true on write, the dns and dns_alt fields are r
    },
    "dns": {
      "example": "8.8.8.8",
      "type": "string",
      "nullable": true,
      "description": "Can be `null` if it's not connected."
    },
    "dns_alt": {
      "example": "8.8.4.4",
      "type": "string",
      "nullable": true,
      "description": "Can be `null` if it's not connected."
    },
    "password": {
      "example": "cGFzc3dvcmQ=",
      "format": "password",
      "type": "string",
      "nullable": true,
      "writeOnly": true,
      "description": "Can be `null` if it's not set."
    }
  },
  "watchdog": {
    "properties": {
      "rwdg_disable": {
        "example": false,
        "type": "boolean",

```

```

        "description": "- `true`: disable Bond Cloud watchdog\n- `false`: enable Bond Cloud watchdog",
    },
    "rwdg_timer_ms": {
        "example": 60000,
        "type": "number",
        "readOnly": true,
        "description": "Number of milliseconds left on watchdog timer before Bond will reboot\n",
    }
},
},
"Ap": {
    "description": "Config AP (soft access point) control settings",
    "properties": {
        "auto": {
            "example": true,
            "type": "boolean",
            "description": "- `true` (default): Config AP follows automatic timer logic.\n  AP is shown as up when timer expires",
        },
        "enabled": {
            "example": false,
            "type": "boolean",
            "description": "Current Config AP state (read), or desired state (write).\n\nIn auto mode, AP is shown as up when timer expires",
        }
    }
},
},
"Eth": {
    "properties": {
        "status": {
            "example": 2,
            "type": "number",
            "description": "- 0 = no link\n- 1 = link, but no IP address yet\n- 2 = link and IP address assigned",
        },
        "mac": {
            "example": "30aea48fa8e8",
            "type": "string",
            "readOnly": true
        },
        "static_ip_set": {
            "example": false,
            "type": "boolean",
            "description": "[changed in v2.16.2]\nIf true on write, the ip, gw, netmask fields are read-only",
        },
        "ip": {
            "example": "192.168.1.42",
            "type": "string"
        },
        "gw": {
            "example": "192.168.1.1",
            "type": "string"
        },
        "netmask": {
            "example": "255.255.255.0",
            "type": "string"
        },
        "dns_set": {
            "example": true,
            "type": "boolean",
            "description": "[changed in v2.16.2]\nIf true on write, the dns and dns_alt fields are read-only",
        },
        "dns": {
            "example": "8.8.8.8",
            "type": "string"
        },
        "dns_alt": {
            "example": "8.8.4.4",
            "type": "string"
        }
    }
},
},
"Progress": {
    "properties": {
        "progress": {
            "example": -1,
            "type": "number",
            "description": "Integer describing status of a firmware upgrade.\n\n-1-999 = upgrade in progress",
        },
        "error_msg": {

```

```

        "example": "net_connect",
        "type": "string",
        "description": "Only present when progress is less than zero.\n\nThis string MAY be displayed\n\n"
    },
    },
    },
    "Upgrade": {
        "properties": {
            "host": {
                "example": "s3.amazonaws.com",
                "type": "string",
                "description": "Fully-Qualified Domain Name or IP address of server from which to download\n\n"
            },
            "port": {
                "example": "443",
                "type": "string",
                "description": "HTTPS port number of server.\nUsed only if `sys/version` is missing key\n\n"
            },
            "http_port": {
                "example": "80",
                "type": "string",
                "description": "HTTP port number of server.\nUsed only if `sys/version` contains true value\n\n"
            },
            "path": {
                "example": "/bond-updates/breck-mach1-v2.0.bin",
                "type": "string",
                "description": "Path of HTTPS URL to use to fetch the firmware binary.\nMust not include\n\n"
            },
            "info": {
                "example": "cc8d372f1b1ada5b99097e46f9d45107487e5f8b000d5ccd943100b3dc27fb55:bond-simulation\n\n",
                "type": "string",
                "description": "Firmware info string, consisting of\n\nSHA-256 hash of the firmware binary\n\n"
            },
            "sig": {
                "example": "PAGliMxpWYYkuG+48JDu6rYfF9V57xXGny2xf8yXCoo2Gcdk8plHrez2bPWkL8TZ5ptaZBPE9UuQ\n\n",
                "type": "string",
                "description": "Digital signature of `info` string, signed using Olibra's private key,\n\n"
            },
            "reboot": {
                "example": 1,
                "type": "number",
                "description": "If non-zero, requests that the Bond reboot spontaneously upon\n\nsuccessful\n\n"
            }
        },
        "required": [
            "host",
            "port",
            "path",
            "info",
            "sig"
        ]
    },
    "Version": {
        "properties": {
            "upgrade_http": {
                "example": true,
                "type": "boolean",
                "description": "If true, Bond will use HTTP protocol using `http_port` specified\n\n in sys\n\n"
            },
            "bondid": {
                "example": "KSMJWCE12345",
                "type": "string",
                "description": "Bond ID (serial number). Unique within the Bond platform."
            },
            "target": {
                "example": "zermatt",
                "type": "string",
                "description": "The target identifies the configuration of bond-core firmware\n\nrunning on\n\n"
            },
            "api": {
                "example": 2,
                "type": "integer",
                "description": "API version increments only when a change occurs in the firmware API\n\n v\n\n"
            },
            "fw_ver": {
                "example": "v2.9",
                "type": "string",
            }
        }
    }
}

```

```

        "description": "Firmware version.\n\nExamples of release versions:\n\n - `v2.2`\n - `v",
    },
    "fw_date": {
        "example": "Wed Nov 22 15:45:19 UTC 2020",
        "type": "string",
        "description": "Human-readable string describing the firmware build date.\n\nFor display",
    },
    "mcu_ver": {
        "example": "v1.234",
        "type": "string",
        "description": "Describes firmware on secondary processor (MCU) of Bond hardware\nwith s",
    },
    "make": {
        "example": "Olibra LLC",
        "type": "string",
        "description": "brand or manufacturer."
    },
    "branding_profile": {
        "example": "BondHome",
        "type": "string",
        "description": "Symbol used for looking up a customized branded experience.\nThis is not",
    },
    "model": {
        "example": "BD-1000",
        "type": "string",
        "description": "Device model number. Displayed to user in advanced settings.\n\nUsed, al",
    }
},
"Reset": {
    "properties": {
        "type": {
            "example": "factory",
            "type": "string",
            "description": "- `\"factory\"`: factory reset (aka `\"White Reset\"`) \n- `\"setup\"`: setu",
        }
    }
},
"Time": {
    "type": "object",
    "properties": {
        "unix_time": {
            "example": 1581708795,
            "type": "number",
            "description": "Absolute time, in seconds since the UNIX epoch.\n\nStarts at zero at fir",
        },
        "time_set": {
            "example": true,
            "type": "boolean",
            "description": "If false, then time has not (yet) been set or determined via NTP.\n\nDev",
        },
        "tz": {
            "example": "America/Sao_Paulo",
            "type": "string",
            "nullable": true,
            "description": "Timezone string, as per tz_database.\n\n`null` if unset.\n\nThe current",
        },
        "grid": {
            "example": "FN30aw",
            "type": "string",
            "nullable": true,
            "description": "Approximate geolocation of Bond unit,\nexpressed as a 6-character Maiden",
        }
    }
},
"Power": {
    "type": "object",
    "properties": {
        "poe_active": {
            "example": true,
            "type": "boolean",
            "description": "True if the unit is being supplied PoE.\n"
        },
        "usb_active": {
            "example": false,
            "type": "boolean",
            "description": "True if the unit is being supplied USB.\n\nNOTE: multiple supplies may b",
        }
    }
}

```



```

    }
  },
  "IndicateSchema": {
    "type": "object",
    "description": "[Mate Pro (MT-1500) only] [v4.26+]\n\nVisual indication animations displayed",
    "properties": {
      "identify": {
        "example": 0,
        "type": "integer",
        "description": "Radar/sonar animation: expanding concentric circles from center.\n\nUsed",
      },
      "commit": {
        "example": 3,
        "type": "integer",
        "description": "Checkmark animation: brief confirmation indicator.\n\nUsed to confirm the",
      }
    }
  },
  "IndicatePatchSchema": {
    "type": "object",
    "description": "Provide exactly one field to trigger that animation.\n",
    "properties": {
      "identify": {
        "example": 10,
        "type": "integer",
        "description": "Radar/sonar animation duration in seconds (0-30).\n"
      },
      "commit": {
        "example": 3,
        "type": "integer",
        "description": "Checkmark animation duration in seconds (0-30).\n"
      }
    }
  },
  "example": {
    "identify": 10
  }
},
"VitalsSchema": {
  "type": "object",
  "description": "Device lifetime statistics. All counters are cumulative since first power-on",
  "properties": {
    "wakes": {
      "type": "object",
      "description": "Wake source counters (from deep sleep).",
      "properties": {
        "acc": {
          "type": "integer",
          "description": "Accelerometer wake-ups followed by user interaction.\nIndicates the",
        },
        "acc_spurious": {
          "type": "integer",
          "description": "Accelerometer wake-ups with no subsequent interaction.\nHigh counts",
        },
        "sked": {
          "type": "integer",
          "description": "Schedule or RTC alarm wake-ups."
        },
        "btn": {
          "type": "integer",
          "description": "Button press wake-ups."
        },
        "ble": {
          "type": "integer",
          "description": "BLE connection-initiated wake-ups."
        }
      }
    }
  },
  "boots": {
    "type": "object",
    "description": "Boot/reset counters by cause.",
    "properties": {
      "other": {
        "type": "integer",
        "description": "Other/unclassified boot reasons."
      },
      "wdt": {

```

```

        "type": "integer",
        "description": "Watchdog timer resets (firmware hang or deadlock).",
    },
    "hardfault": {
        "type": "integer",
        "description": "Hard fault resets (CPU exception).",
    },
    "powercycle": {
        "type": "integer",
        "description": "Cold power cycle boots.",
    },
    "bor": {
        "type": "integer",
        "description": "Brown-out resets (low voltage).",
    },
    "menu": {
        "type": "integer",
        "description": "User-initiated reboots via on-device menu.",
    },
    "api": {
        "type": "integer",
        "description": "API-triggered reboots (sys/reboot).",
    }
    },
    "actions": {
        "type": "object",
        "description": "User action counters (transmissions).",
        "properties": {
            "btn": {
                "type": "integer",
                "description": "Single-channel button press actions.",
            },
            "btn_ch00": {
                "type": "integer",
                "description": "All-channel (channel 0) button press actions.",
            },
            "sked": {
                "type": "integer",
                "description": "Schedule-triggered actions.",
            },
            "api": {
                "type": "integer",
                "description": "API-triggered actions.",
            }
        }
    },
    "display_s": {
        "type": "object",
        "description": "Display-on time in seconds by brightness level.",
        "properties": {
            "high": {
                "type": "integer",
                "description": "Seconds at high brightness (outdoor conditions).",
            },
            "low": {
                "type": "integer",
                "description": "Seconds at low brightness (indoor conditions).",
            }
        }
    },
    "ble": {
        "type": "object",
        "description": "BLE connection statistics.",
        "properties": {
            "conns": {
                "type": "integer",
                "description": "Total BLE connection count.",
            },
            "conn_s": {
                "type": "integer",
                "description": "Total BLE connected time in seconds.",
            }
        }
    },
    "wipes": {
        "type": "integer",

```

```

        "description": "Factory reset count. Persists across resets."
    },
    "seconds": {
        "type": "integer",
        "description": "Total powered time in seconds. Serves as an odometer for the device.\nUpd
    },
    "sleep_s": {
        "type": "integer",
        "description": "Total sleep time in seconds. Cumulative time in deep sleep as measured\n
    },
    "timestamp_first": {
        "type": "number",
        "description": "Unix timestamp when the vitals record was created or last reset."
    },
    "timestamp_last": {
        "type": "number",
        "description": "Unix timestamp of the last vitals save (updated before each deep sleep).
    }
},
"example": {
    "wakes": {
        "acc": 142,
        "acc_spurious": 8,
        "btn": 87
    },
    "boots": {
        "other": 1,
        "powercycle": 3,
        "api": 2
    },
    "actions": {
        "btn": 298,
        "btn_ch00": 14,
        "sked": 7
    },
    "display_s": {
        "high": 3600,
        "low": 7200
    },
    "ble": {
        "conns": 42,
        "conn_s": 1800
    },
    "wipes": 1,
    "seconds": 86400,
    "sleep_s": 72000,
    "timestamp_first": 1709510400,
    "timestamp_last": 1709596800
},
"Backup": {
    "properties": {
        "host": {
            "example": "192.168.1.107",
            "type": "string",
            "description": "IP address of local HTTP server that will accept or serve snapshot.\n"
        },
        "http_port": {
            "example": "80",
            "type": "string",
            "description": "HTTP port number of server.\nUsually not 80 because mobile phone or other
        },
        "path": {
            "example": "/snapshots",
            "type": "string",
            "description": "Path to directory on HTTP server to use.\nMust not include any query or
        },
        "timestamp": {
            "example": "1618599782",
            "type": "string",
            "description": "Required only for backup operation.\nTimestamp for inclusion in the file
        },
        "filename": {
            "example": "/ZZDE12345_v2.18.4-alpha_1618599782_0f8d01fa.bsnap",
            "type": "string",
            "description": "Required only for restore operation.\nFirmware generates the filename for
        },
    },

```

```

    "dev_ids": {
      "example": [
        "aabbccdd"
      ],
      "type": "string",
      "description": "If not provided, all devices are included in backup.\n\nIf provided, this"
    },
    "backup": {
      "example": 0,
      "type": "number",
      "readOnly": true,
      "description": "- `0`: no backup running\n- `1`: backup still running\n- `2`: backup succe"
    },
    "restore": {
      "example": -1,
      "type": "number",
      "readOnly": true,
      "description": "- `0`: no restore running\n- `1`: restore still running\n- `2`: restore s"
    },
    "progress": {
      "example": 999,
      "type": "number",
      "readOnly": true,
      "description": "Number from 0-1000 representing progress of `backup` or `restore`. \nProgre"
    },
    "error_msg": {
      "example": "connection error",
      "type": "string",
      "readOnly": true,
      "description": "Human-readable description of why `backup` or `restore` is `-1`. \nAbsent"
    }
  }
},
"weatherSensor": {
  "type": "object",
  "properties": {
    "_id": {
      "writeOnly": true,
      "example": "WEAXX12345",
      "type": "string",
      "description": "Provided only on POST request to specify the Weather Sensor ID. \n"
    },
    "name": {
      "example": "My Weather Sensor",
      "type": "string",
      "description": "Name of Weather Sensor on Bond Home app and suggested name for \ncontrol"
    },
    "location": {
      "example": "My garden",
      "type": "string",
      "description": "Location of Weather Sensor. \n"
    },
    "event_links": {
      "example": {
        "1": [
          {
            "device": "xxyyzzww",
            "action": "TurnOn"
          }
        ],
        "2": [
          {
            "device": "aabbccdd",
            "action": "SetPosition",
            "argument": 75
          }
        ],
        "3": []
      },
      "type": "object",
      "description": "Object describes every link between weather events from this Weather Sen"
    },
    "signal": {
      "readOnly": true,
      "example": 97,
      "type": "integer",
      "description": "Signal quality indication, range 1-100. \nNote that, to avoid hash thrash"
    }
  }
},

```

```

    },
    "battery": {
      "readOnly": true,
      "example": 80,
      "type": "integer",
      "description": "Weather Sensor battery remaining in percentage, range 1-100.\n"
    },
    "model": {
      "readOnly": true,
      "example": "BWS-1000",
      "type": "string",
      "description": "User-facing model number of Weather Sensor.\n"
    },
    "type": {
      "readOnly": true,
      "example": "weather_sensor",
      "type": "string",
      "description": "Indicates the type of Sidekick, always return `weather_sensor`.\n"
    }
  }
},
"schemas_Properties": {
  "type": "object",
  "properties": {
    "feature_wind": {
      "example": true,
      "type": "boolean",
      "description": "Enable or disable the wind detection functionality.\n"
    },
    "feature_rain": {
      "example": true,
      "type": "boolean",
      "description": "Enable or disable the rain detection functionality.\n"
    },
    "feature_sun": {
      "example": true,
      "type": "boolean",
      "description": "Enable or disable the sunlight intensity detection functionality.\n"
    },
    "feature_wind_push": {
      "example": true,
      "type": "boolean",
      "description": "Enable or disable the push notification for when the wind speed approach\n"
    },
    "wind_threshold_dms": {
      "example": 85,
      "type": "integer",
      "description": "Wind detection threshold in decimeters/second. \n"
    },
    "sun_threshold_level": {
      "example": 5,
      "type": "integer",
      "description": "Sun level threshold. From 1 (lowest intensity) to 8 (highest intensity).\n"
    },
    "wind_ignore": {
      "example": true,
      "type": "boolean",
      "description": "Ignore wind detections for a specified amount of time. \nWhen setting to\n"
    },
    "wind_ignore_expiration": {
      "example": 3600,
      "type": "integer",
      "description": "Number of seconds for ignoring wind detections.\n"
    }
  }
},
"schemas_State-2": {
  "type": "object",
  "properties": {
    "status": {
      "example": "triggered_wind",
      "type": "string",
      "readOnly": true,
      "description": "String indicating the current status of the Weather Sensor. \nPossible v\n"
    },
    "data_wind_speed_dms": {
      "example": 32,

```

```

    "type": "integer",
    "readOnly": true,
    "description": "Last measurement of wind speed in decimeters/second.\n"
  },
  "data_rain_mmh": {
    "example": 2,
    "type": "integer",
    "readOnly": true,
    "description": "Last measurement of rain rate in milimeters/hour.\n"
  },
  "data_sun_level": {
    "example": 6,
    "type": "integer",
    "readOnly": true,
    "description": "Last measurement of sun level.\n"
  },
  "data_unixtime": {
    "example": 1696479511,
    "type": "integer",
    "readOnly": true,
    "description": "Unixtime of the last data measurements.\n"
  },
  "is_raining": {
    "example": false,
    "type": "boolean",
    "readOnly": true,
    "description": "Flag indicating if it is currently raining or not.\n"
  },
  "battery_voltage_dV": {
    "example": 24,
    "type": "integer",
    "readOnly": true,
    "description": "Current state of the Weather Sensor solar battery in decivolts.\n"
  },
  "battery_2_voltage_dV": {
    "example": 24,
    "type": "integer",
    "readOnly": true,
    "description": "Current state of the Weather Sensor backup battery (AA cells) in decivolts.\n"
  },
  "battery": {
    "example": 80,
    "type": "integer",
    "readOnly": true,
    "description": "Current state of the Weather Sensor solar battery in percentage, range 1-100.\n"
  },
  "battery_2": {
    "example": 80,
    "type": "integer",
    "readOnly": true,
    "description": "Current state of the Weather Sensor backup battery in percentage, range 1-100.\n"
  },
  "status_flag_unstable": {
    "type": "boolean",
    "example": false,
    "readOnly": true,
    "description": "Flag indicating if Weather Sensor data rate is lower than expected.\n"
  },
  "status_flag_no_data": {
    "type": "boolean",
    "example": false,
    "readOnly": true,
    "description": "Flag indicating if Weather Sensor data is missing for more than 30 minutes.\n"
  },
  "status_flag_battery_low": {
    "type": "boolean",
    "example": false,
    "readOnly": true,
    "description": "Flag indicating if solar battery is running low.\n"
  },
  "status_flag_battery_2_low": {
    "example": true,
    "type": "boolean",
    "readOnly": true,
    "description": "Flag indicating if backup battery (AA cells) is running low.\n"
  },
  "status_flag_low_temperature": {

```

```

        "example": true,
        "type": "boolean",
        "readOnly": true,
        "description": "Flag indicating if Weather Sensor temperature is low, recommending opera
    },
    "data_temperature_dc": {
        "example": 212,
        "type": "integer",
        "readOnly": true,
        "description": "Last measurement of the temperature in deci-Celsius.\n"
    },
    "data_humidity_percent": {
        "example": 65,
        "type": "integer",
        "readOnly": true,
        "description": "Last measurement of the humidity in percentage (0 to 100).\n"
    }
},
"ChannelList": {
    "type": "object",
    "description": "Hash tree of all channels on this Mate Pro.\nChannel IDs range from \"0\" to
    "example": {
        "0": {
            "_": "a1b2c3d4"
        },
        "1": {
            "_": "84819a9f"
        },
        "2": {
            "_": "23141efa"
        },
        "3": {
            "_": "2425a8bc"
        },
        "4": {
            "_": "2425a8bc"
        },
        "5": {
            "_": "2425a8bc"
        },
        "6": {
            "_": "2425a8bc"
        },
        "7": {
            "_": "2425a8bc"
        },
        "8": {
            "_": "2425a8bc"
        },
        "9": {
            "_": "2425a8bc"
        },
        "10": {
            "_": "2425a8bc"
        },
        "11": {
            "_": "2425a8bc"
        },
        "12": {
            "_": "2425a8bc"
        },
        "13": {
            "_": "2425a8bc"
        },
        "14": {
            "_": "2425a8bc"
        },
        "15": {
            "_": "2425a8bc"
        },
        "_": "7fc1e84b"
    }
},
"Channel": {
    "type": "object",
    "properties": {

```

```

    "name": {
      "example": "North Screen",
      "type": "string",
      "description": "User-facing name of the channel, displayed in the app.\nMaximum length: 64",
    },
    "label": {
      "example": "N Screen",
      "type": "string",
      "description": "[MT-1500 only]\nOptional display label shown on the device's LED matrix.",
    },
    "template": {
      "example": "RMS12",
      "type": "string",
      "description": "Technology template assigned to this channel (R-number identifier).\nDetermines the technology-specific properties for this channel.",
    },
    "properties": {
      "type": "object",
      "description": "Technology-specific properties for this channel.\nContents vary based on the technology template assigned to this channel.",
      "properties": {
        "mfg": {
          "example": 0,
          "type": "integer",
          "description": "Manufacturer code or technology identifier.\n",
        },
        "pairing_assets_key": {
          "example": "",
          "type": "string",
          "description": "Reference to pairing assets or instructions.\n",
        }
      }
    },
    "example": {
      "mfg": 0,
      "pairing_assets_key": ""
    },
    "type": {
      "readOnly": true,
      "example": "MS",
      "type": "string",
      "description": "[v4.24+] [MT-1500 only]\nChannel type, automatically inferred from the channel's technology template.",
    },
    "enabled": {
      "example": true,
      "type": "boolean",
      "description": "[v4.24+] [MT-1500 only]\nChannel visibility toggle. When false, the channel is hidden from the app.",
    },
    "actions": {
      "readOnly": true,
      "type": "array",
      "items": {
        "type": "string"
      },
      "example": [
        "Raise",
        "Lower",
        "Stop",
        "SetPosition"
      ],
      "description": "[v4.24+] [MT-1500 only]\nList of actions available on this channel, determined by the channel's technology template.",
    }
  },
  "keyConfig": {
    "type": "object",
    "properties": {
      "mode": {
        "type": "string",
        "enum": [
          "cta",
          "goto",
          "keystream"
        ],
        "description": "Button operation mode:\n\n- `cta`: Call To Action mode (default). Pressing the button triggers the action specified in the `cta` field of the channel's `properties` object.\n- `goto`: Goto mode. Pressing the button triggers the action specified in the `goto` field of the channel's `properties` object.\n- `keystream`: Keystream mode. Pressing the button triggers a continuous stream of keystrokes specified in the `keystream` field of the channel's `properties` object.",
      },
      "channel": {
        "type": "integer",
        "description": "Target channel for goto mode (required when mode=goto).\n\nValid values: 0-15",
      }
    }
  }
}

```



```

        "example": 5
    },
    "required": [
        "mode"
    ],
    "example": {
        "mode": "goto",
        "channel": 5
    }
},
"KeyConfiguration": {
    "type": "object",
    "description": "Configuration for the three Quick Action Buttons (K1, K2, K3) on Mate Pro (M",
    "properties": {
        "1": {
            "$ref": "#/components/schemas/KeyConfig"
        },
        "2": {
            "$ref": "#/components/schemas/KeyConfig"
        },
        "3": {
            "$ref": "#/components/schemas/KeyConfig"
        }
    },
    "example": {
        "1": {
            "mode": "cta"
        },
        "2": {
            "mode": "keystream"
        },
        "3": {
            "mode": "goto",
            "channel": 15
        }
    }
},
"responses": {
    "Unauthorized": {
        "content": {
            "application/json": {
                "schema": {
                    "$ref": "#/components/schemas/Unauthorized"
                }
            }
        },
        "description": "Unauthorized"
    },
    "NotFound": {
        "content": {
            "application/json": {
                "schema": {
                    "$ref": "#/components/schemas/NotFound"
                }
            }
        },
        "description": "Not Found"
    },
    "InternalServerError": {
        "content": {
            "application/json": {
                "schema": {
                    "$ref": "#/components/schemas/InternalServerError"
                }
            }
        },
        "description": "Internal Server Error"
    },
    "Created": {
        "content": {
            "application/json": {
                "schema": {
                    "$ref": "#/components/schemas/Created"
                }
            }
        }
    }
}

```

```

    },
    "description": "Resource Created"
  },
  "BadRequest": {
    "content": {
      "application/json": {
        "schema": {
          "$ref": "#/components/schemas/BadRequest"
        }
      }
    },
    "description": "Bad request"
  },
  "Conflict": {
    "content": {
      "application/json": {
        "schema": {
          "$ref": "#/components/schemas/Conflict"
        }
      }
    },
    "description": "Conflict"
  },
  "Locked": {
    "content": {
      "application/json": {
        "schema": {
          "$ref": "#/components/schemas/Locked"
        }
      }
    },
    "description": "Resource is Locked"
  }
}
}
}
}

```